# Python Algorithm Library

## Contents


**1 Templates** **2**
- 1.1 ICPCtemplate.py . . . 2

**2 gen** **2**
- 2.1 graph.py . . . 2

**3 data-structure** **2**
- 3.1 2DSparseTable.py . . . 2
- 3.2 BIT.py . . . 2
- 3.3 BucketList.py . . . 2
- 3.4 CartesianTree.py . . . 3
- 3.5 DisjointSparseTable.py . . . 3
- 3.6 LazySegtree.py . . . 3
- 3.7 LinearImos.py . . . 4
- 3.8 MargeSortTree.py . . . 5
- 3.9 Mos_algorithm.py . . . 5
- 3.10 PackedTupleHeap.py . . . 5
- 3.11 Segtree.py . . . 6
- 3.12 SortedMultiset.py . . . 6
- 3.13 SortedSet.py . . . 7
- 3.14 SparseTable.py . . . 7
- 3.15 TopK.py . . . 8
- 3.16 TupleLazySegtree.py . . . 8
- 3.17 TupleSegtree.py . . . 9
- 3.18 TwoWayList.py . . . 9

**4 graph** **10**
- 4.1 Bellman_ford.py . . . 10
- 4.2 BipartiteGraph.py . . . 10
- 4.3 EulerTour.py . . . 10
- 4.4 LCA.py . . . 11
- 4.5 PotentialUnionFind.py . . . 11
- 4.6 Rerooting.py . . . 12
- 4.7 SCC.py . . . 12
- 4.8 TopologicalSort.py . . . 13
- 4.9 UnionFind.py . . . 13
- 4.10 WarshallFloyd.py . . . 13
- 4.11 cycle_detect.py . . . 13
- 4.12 maxflow.py . . . 13

**5 hash** **14**
- 5.1 MultisetHash.py . . . 14
- 5.2 ZobristHash.py . . . 14
- 5.3 tupleToHash.py . . . 14

**6 math** **14**
- 6.1 AngularSort.py . . . 14
- 6.2 FFT.py . . . 14
- 6.3 MatrixPower.py . . . 15
- 6.4 PIE.py . . . 16
- 6.5 Polygon.py . . . 16
- 6.6 Polygon_int.py . . . 16
- 6.7 Triangle.py . . . 16
- 6.8 calc.py . . . 17
- 6.9 circle_int.py . . . 17
- 6.10 extGCD.py . . . 18
- 6.11 factorization.py . . . 18
- 6.12 geometry.py . . . 18
- 6.13 line_int.py . . . 18
- 6.14 mobius.py . . . 19
- 6.15 modComb.py . . . 19
- 6.16 modDiv.py . . . 20

**7 string** **20**
- 7.1 RollingHash.py . . . 20
- 7.2 SuffixArray.py . . . 20
- 7.3 Z-algo.py . . . 20
- 7.4 changeable_RollingHash.py . . . 21
- 7.5 sort_by_concat.py . . . 21

**8 techniques** **22**
- 8.1 3buntansaku.py . . . 22
- 8.2 CmpSort.py . . . 22
- 8.3 Compression.py . . . 22
- 8.4 CrossAccumulate.py . . . 22
- 8.5 Doubling.py . . . 22
- 8.6 MegruBisect.py . . . 23
- 8.7 Rotate.py . . . 23
- 8.8 RunLengthCompression.py . . . 23
- 8.9 c_is_buween_a_to_b.py . . . 23
- 8.10 imos.py . . . 23

# 1 Templates

## 1.1 ICPCtemplate.py

```python
import sys
def input():return sys.stdin.readline().rstrip()

# 入力など
def ii(): return int(input())
def ms(): return map(int, input().split())
def li(): return list(map(int,input().split()))
inf = pow(10,18)
mod = 998244353

# ライブラリ系
# import pypyjit
# pypyjit.set_param('max_unroll_recursion=-1')
sys.setrecursionlimit(10**9)
from collections import defaultdict,deque
from itertools import combinations,permutations,accumulate,product
from bisect import bisect_left,bisect_right
from heapq import heappop,heappush,heapify
from functools import cache
# @cache
```

# 2 gen

## 2.1 graph.py

```python
# 単純連結無向グラフの生成（1-indexed）
from random import randint, shuffle

N = randint(3, 10)
V = list(range(1, N + 1))
shuffle(V)
M = []
for i in range(1, N):
    p = randint(0, i - 1)  # 前の頂点から必ず 1 つつなぐ
    for j in range(i):
        if j == p or randint(0, 1):
            M.append((V[j], V[i]))

print(N, len(M))
for e in M:
    print(*e)
```

# 3 data-structure

## 3.1 2DSparseTable.py

```python
# 未 verified
# メモリ使用量に注意


class SparseTable2D:
    def __init__(self, grid, op):
        """
        grid: 2 次元配列
        op: 演算（結合則、冪等性が必要）
        O(H * W * logH * logW)
        """
        self.op = op
        self.H = len(grid)
        self.W = len(grid[0])
        self.K = self.H.bit_length()
        self.L = self.W.bit_length()

        # table[k][l][i][j]: 左上 (i, j)、高さ 2^k、幅 2^l の矩形に op を適用した結果
        self.table = [
            [[[0] * self.W for _ in range(self.H)] for _ in range(self.L)]
            for _ in range(self.K)
        ]

        for i in range(self.H):
            for j in range(self.W):
                self.table[0][0][i][j] = grid[i][j]

        # 行方向の構築（高さ 1 固定で幅を拡大）
        for l in range(1, self.L):
            width = 1 << l
            half = width >> 1
            for i in range(self.H):
                # j から幅 2^l の区間が収まる範囲だけ計算
                for j in range(self.W - width + 1):
                    self.table[0][l][i][j] = op(
                        self.table[0][l - 1][i][j], self.table[0][l - 1][i][j + half]
                    )
        # 列方向の構築（任意の幅 l に対して高さを拡大）
        for k in range(1, self.K):
            height = 1 << k
            half = height >> 1
            for l in range(self.L):
                width = 1 << l
                for i in range(self.H - height + 1):
                    for j in range(self.W - width + 1):
                        self.table[k][l][i][j] = op(
                            self.table[k - 1][l][i][j],
                            self.table[k - 1][l][i + half][j],
                        )

    def query(self, r1, c1, r2, c2):
        """
        (r1, c1) を左上, (r2, c2) を右下とする閉区間矩形の演算結果を返す
        0 <= r1 <= r2 < R, 0 <= c1 <= c2 < C
        """
        assert 0 <= r1 <= r2 < self.H
        assert 0 <= c1 <= c2 < self.W

        height = r2 - r1 + 1
        width = c2 - c1 + 1
        k = height.bit_length() - 1  # 2^k <= height を満たす最大の k
        l = width.bit_length() - 1  # 2^l <= width を満たす最大の l

        r2_start = r2 - (1 << k) + 1
        c2_start = c2 - (1 << l) + 1

        # 4 つの長方形で (r1,c1)-(r2,c2) をカバーする
        tl = self.table[k][l][r1][c1]
        tr = self.table[k][l][r1][c2_start]
        bl = self.table[k][l][r2_start][c1]
        br = self.table[k][l][r2_start][c2_start]

        return self.op(self.op(tl, tr), self.op(bl, br))
```

## 3.2 BIT.py

```python
class BIT:
    def __init__(self, n):
        self.n = n
        self.bit = [[0] * self.n for _ in range(2)]

    def add(self, l, r, x):  # [l,r) に x を加算
        bit0, bit1 = self.bit
        n = self.n
        c = -x * l
        while l < n:
            bit0[l] += c
            bit1[l] += x
            l |= l + 1
        c = x * r
        while r < n:
            bit0[r] += c
            bit1[r] -= x
            r |= r + 1

    # 区間 [0, i) の和を取得
    def SUM(self, i):
        bit0, bit1 = self.bit
        s0 = s1 = 0
        idx = i - 1
        while idx >= 0:
            s0 += bit0[idx]
            s1 += bit1[idx]
            idx = (idx & (idx + 1)) - 1
        return s0 + s1 * i

    # 区間 [l, r) の和を取得
    def range_sum(self, l, r):
        return self.SUM(r) - self.SUM(l)

    # a[0] + ... + a[x] >= w となる最小の x を O(log n) で求める
    # 各要素は非負であること。w <= 0 なら 0、見つからなければ n
    def lower_bound(self, w):
        if w <= 0:
            return 0

        bit0, bit1 = self.bit
        n = self.n
        x = -1
        s0 = s1 = 0
        step = 1 << n.bit_length()
        while step:
            nxt = x + step
            if nxt < n:
                t0 = s0 + bit0[nxt]
                t1 = s1 + bit1[nxt]
                # SUM(nxt + 1) を 2 本の BIT の累積値から復元する
                if t0 + t1 * (nxt + 1) < w:
                    x, s0, s1 = nxt, t0, t1
            step >>= 1
        return x + 1
```

## 3.3 BucketList.py

```python
# 任意の場所に挿入したり削除とかができる
import math


class BucketList:
    br = 16
    sr = 24

    def __init__(s, a=[]):
        a = list(a)
        n = s.size = len(a)
        k = math.ceil(math.sqrt(n / s.br))
        s.a = [
            a[n * i // k : n * (i + 1) // k]
            for i in range(k)
        ]

    def __iter__(s):
        for a in s.a:
            yield from a

    def __contains__(s, x):
        "x が含まれていれば True を返す。/ O(N)"
        for y in s:
            if x == y:
                return True
        return False
```

```

    def _insert(s, a, b, i, x):
        a.insert(i, x)
        s.size += 1
        if len(a) > len(s.a) * s.sr:
            mid = len(a) >> 1
            s.a[b : b + 1] = [a[:mid], a[mid:]]

    def insert(s, i, x):
        "i 番目の位置に x を挿入する。/ O(√ N)"
        if s.size == 0:
            if i != 0 and i != -1:
                raise IndexError
            s.a = [[x]]
            s.size = 1
            return
        if i < 0:
            for b, a in enumerate(reversed(s.a)):
                i += len(a)
                if i >= 0:
                    return s._insert(a, len(s.a) + ~b, i, x)
        else:
            for b, a in enumerate(s.a):
                if i <= len(a):
                    return s._insert(a, b, i, x)
                i -= len(a)
        raise IndexError

    def append(s, x):
        "末尾に x を追加する。/ 償却 O(1)"
        if s.size == 0:
            s.a = [[x]]
            s.size = 1
            return
        a = s.a[-1]
        return s._insert(a, len(s.a) - 1, len(a), x)

    def extend(s, a):
        for x in a:
            s.append(x)

    def __getitem__(s, i):
        if i < 0:
            for a in reversed(s.a):
                i += len(a)
                if i >= 0:
                    return a[i]
        else:
            for a in s.a:
                if i < len(a):
                    return a[i]
                i -= len(a)
        raise IndexError

    def _pop(s, a, b, i):
        ans = a.pop(i)
        s.size -= 1
        if not a:
            del s.a[b]
        return ans

    def pop(s, i=-1):
        "i 番目の要素を削除して返す。/ O(√ N) / i < 0 なら O(-i)"
        if i < 0:
            for b, a in enumerate(reversed(s.a)):
                i += len(a)
                if i >= 0:
                    return s._pop(a, ~b, i)
        else:
            for b, a in enumerate(s.a):
                if i < len(a):
                    return s._pop(a, b, i)
                i -= len(a)
        raise IndexError

    def count(s, x):
        "x の出現回数を返す。/ O(N)"
        return sum(1 for y in s if x == y)

    def index(s, x):
        "最初に現れる x の添字を返す。見つからなければ ValueError を送出する。/ O(N)"
        for i, y in enumerate(s):
            if x == y:
                return i
        raise ValueError

    def remove(s, x):
        "最初に現れる x を削除する。見つからなければ ValueError を送出する。/ O(N)"
        s.pop(s.index(x))

    def clear(s):
        s.a = []
        s.size = 0

    def reverse(s):
        s.a.reverse()
        for a in s.a:
            a.reverse()

    def copy(s):
        return BucketList(s)

    def __len__(s):
        return s.size

    def __str__(s):
        return str(list(s))
```

## 3.4 CartesianTree.py

```
class CartesianTree:
    def __init__(self, A):
        n = len(A)
        self.root = 0
        self.left = [-1] * n
        self.right = [-1] * n
        self.parent = [-1] * n
        st = [-1] * n
        st[0] = 0
        st_top = 0
        for i in range(1, n):
            if A[st[st_top]] > A[i]:  # pop してから push
                while st_top >= 1 and A[st[st_top - 1]] > A[i]:
                    st_top -= 1
                # st_top > 0 なら A[st[top-1]] <= A[i] < A[st[top]]
                # st[st_top] を左の子、st[st_top-1] を親にする
                self.left[i] = st[st_top]
                self.parent[self.left[i]] = i
                if st_top == 0:
                    self.root = i
                else:
                    self.parent[i] = st[st_top - 1]
                    self.right[self.parent[i]] = i
                st[st_top] = i
            else:  # push するだけ
                self.parent[i] = st[st_top]
                self.right[self.parent[i]] = i
                st_top += 1
                st[st_top] = i
```

## 3.5 DisjointSparseTable.py

```
class DisjointSparseTable:
    def __init__(self, arr, op):
        """
        arr: 配列
        op: 演算 (結合律を満たすこと)
        """
        self.n = len(arr)
        self.arr = arr[:]
        self.op = op

        self.max_k = self.n.bit_length()
        self.table = [[0] * self.n for _ in range(self.max_k)]

        for k in range(self.max_k):
            block_size = 1 << (k + 1)  # ブロック全体の長さ
            half = 1 << k  # 半分の長さ

            for start in range(0, self.n, block_size):
                mid = min(start + half, self.n)
                end = min(start + block_size, self.n)

                # 左側 [start, mid) の累積を計算
                if start < mid:
                    cur = arr[mid - 1]
                    self.table[k][mid - 1] = cur
                    for i in range(mid - 2, start - 1, -1):
                        cur = op(arr[i], cur)
                        self.table[k][i] = cur

                # 右側 [mid, end) の累積を計算
                if mid < end:
                    cur = arr[mid]
                    self.table[k][mid] = cur
                    for i in range(mid + 1, end):
                        cur = op(cur, arr[i])
                        self.table[k][i] = cur

    def query(self, l, r):
        assert 0 <= l < r <= self.n
        if r - l == 1:
            return self.arr[l]

        # クエリに答えるために使うレベル k を求める
        k = (l ^ (r - 1)).bit_length() - 1
        return self.op(self.table[k][l], self.table[k][r - 1])
```

## 3.6 LazySegtree.py

```
class lazy_segtree:  # すべて 0-index
    def _push(s, k):
        lz = s.lz
        f = lz[k]
        # 初期状態・伝播済みのタグは何もしない
        if f is s.ide:
            return
        d, mp, comp = s.d, s.mp, s.comp
        c = 2 * k
        d[c] = mp(f, d[c])
        d[c + 1] = mp(f, d[c + 1])
        if c < s.z:
            lz[c] = comp(f, lz[c])
            lz[c + 1] = comp(f, lz[c + 1])
        lz[k] = s.ide

    def __init__(s, v, op, e, mp, comp, ide):
        s.n = len(v)
        s.log = (s.n - 1).bit_length()
        z = s.z = 1 << s.log
        d = s.d = [e] * (2 * z)
        s.lz = [ide] * z
        s.e = e
        s.op = op
        s.mp = mp
        s.comp = comp
```

```
        s.ide = ide
        d[z : z + s.n] = v
        for i in range(z - 1, 0, -1):
            d[i] = op(d[2 * i], d[2 * i + 1])

    # 一点更新
    def set(s, p, x):
        assert 0 <= p < s.n
        d, op, push = s.d, s.op, s._push
        p += s.z
        for i in range(s.log, 0, -1):
            push(p >> i)
        d[p] = x
        p >>= 1
        while p:
            d[p] = op(d[2 * p], d[2 * p + 1])
            p >>= 1

    # d[p] を取得する
    def get(s, p):
        assert 0 <= p < s.n
        push = s._push
        p += s.z
        for i in range(s.log, 0, -1):
            push(p >> i)
        return s.d[p]

    # 区間 [l,r) の演算結果を返す
    def prod(s, l, r):
        assert 0 <= l <= r <= s.n
        if l == r:
            return s.e
        d, op, push = s.d, s.op, s._push
        l += s.z
        r += s.z
        # 境界が揃わず、伝播・再計算が必要になる最初の段
        left = (l & -l).bit_length()
        right = (r & -r).bit_length()
        for i in range(s.log, 0, -1):
            if i >= left:
                push(l >> i)
            if i >= right:
                push(r >> i)
        sml = smr = s.e
        while l < r:
            if l & 1:
                sml = op(sml, d[l])
                l += 1
            if r & 1:
                smr = op(d[r - 1], smr)
            l >>= 1
            r >>= 1
        return op(sml, smr)

    # 全区間の演算結果を返す
    def all_prod(s):
        return s.d[1]

    # 一点に対して操作する
    def apply_point(s, p, f):
        assert 0 <= p < s.n
        d, op, push = s.d, s.op, s._push
        p += s.z
        for i in range(s.log, 0, -1):
            push(p >> i)
        d[p] = s.mp(f, d[p])
        p >>= 1
        while p:
            d[p] = op(d[2 * p], d[2 * p + 1])
            p >>= 1

    # 区間操作
    # f は mp に与える引数の f
    def apply(s, l, r, f):
        assert 0 <= l <= r <= s.n
        if l == r:
            return
        d, lz = s.d, s.lz
        op, mp, comp = s.op, s.mp, s.comp
        push, z = s._push, s.z
        l += z
        r += z
        # 境界が揃わず、伝播・再計算が必要になる最初の段
        left = (l & -l).bit_length()
        right = (r & -r).bit_length()
        for i in range(s.log, 0, -1):
            if i >= left:
                push(l >> i)
            if i >= right:
                push(r >> i)
        l2, r2 = l, r
        while l < r:
            if l & 1:
                d[l] = mp(f, d[l])
                if l < z:
                    lz[l] = comp(f, lz[l])
                l += 1
            if r & 1:
                r -= 1
                d[r] = mp(f, d[r])
                if r < z:
                    lz[r] = comp(f, lz[r])
            l >>= 1
            r >>= 1
        for i in range(1, s.log + 1):
            if i >= left:
                k = l2 >> i
                d[k] = op(d[2 * k], d[2 * k + 1])
            if i >= right:
                k = r2 >> i
                d[k] = op(d[2 * k], d[2 * k + 1])

    # check(operate(data[l],data[l+1],...,data[r-1])) = True
    # を満たす最大の r を返す
    def max_right(s, l, check):
        assert 0 <= l <= s.n
        assert check(s.e)
        if l == s.n:
            return s.n
        d, op, push = s.d, s.op, s._push
        z = s.z
        l += z
        for i in range(s.log, 0, -1):
            push(l >> i)
        sm = s.e
        while True:
            while l % 2 == 0:
                l >>= 1
            nxt = op(sm, d[l])
            if not check(nxt):
                while l < z:
                    push(l)
                    l *= 2
                    nxt = op(sm, d[l])
                    if check(nxt):
                        sm = nxt
                        l += 1
                return l - z
            sm = nxt
            l += 1
            if (l & -l) == l:
                return s.n

    # check(operate(data[l],data[l+1],...,data[r-1])) = True
    # を満たす最小の l を返す
    def min_left(s, r, check):
        assert 0 <= r <= s.n
        assert check(s.e)
        if r == 0:
            return 0
        d, op, push = s.d, s.op, s._push
        z = s.z
        r += z
        for i in range(s.log, 0, -1):
            push((r - 1) >> i)
        sm = s.e
        while True:
            r -= 1
            while r > 1 and (r % 2):
                r >>= 1
            nxt = op(d[r], sm)
            if not check(nxt):
                while r < z:
                    push(r)
                    r = 2 * r + 1
                    nxt = op(d[r], sm)
                    if check(nxt):
                        sm = nxt
                        r -= 1
                return r + 1 - z
            sm = nxt
            if (r & -r) == r:
                return 0

    # print(seg) で遅延を反映した要素を表示 (O(N log N))
    def __str__(s):
        return str([s.get(i) for i in range(s.n)])


# data の単位元
e = 0
# lazy の単位元
# 区間更新なら lazy としてとりえない値にする
ide = 0


# 区間に対して行いたい演算
def op(a, b):
    return a + b


# 遅延させていた操作を data に伝搬させる関数
# f が遅延させた操作、x が data
# ex. 区間加算なら f+x
def mp(f, x):
    # 区間更新なら
    # if f==ide: return x
    return f + x


# 既に遅延させていた操作にさらに操作を追加する関数
# f が追加する操作、g がいままでの操作
def comp(f, g):
    # 区間更新なら
    # if f==ide: return g
    return f + g
```

## 3.7 LinearImos.py

```
# 区間 [l,r] に i \in [l,r] に一次関数 a*i + b を加算できるライブラリ
# クエリ l, r, a, b：各 i ∈ [l, r] に a*i + b を加算する
class LinearImos:
    def __init__(self, n, mod=None):
        self.n = n
        self.mod = mod
        self.coef = [0] * (n + 1)  # i の係数
        self.cons = [0] * (n + 1)  # 定数項

    def add_linear(self, l, r, a, b):
        """
        閉区間 [l, r] に a*i + b を加算する
```

```
        """
        if l > r:
            return

        if self.mod is None:
            self.coef[l] += a
            self.coef[r + 1] -= a
            self.cons[l] += b
            self.cons[r + 1] -= b
        else:
            mod = self.mod
            self.coef[l] = (self.coef[l] + a) % mod
            self.coef[r + 1] = (self.coef[r + 1] - a) % mod
            self.cons[l] = (self.cons[l] + b) % mod
            self.cons[r + 1] = (self.cons[r + 1] - b) % mod

    def add_arithmetic(self, l, r, start, diff):
        """
        初項 start 公差 diff の等差数列を [l,r] に足す
        閉区間 [l, r] に
        start, start+diff, start+2*diff, ...
        を加算する

        位置 i に足す値は
        start + diff*(i-l)
        = diff*i + (start - diff*l)
        """
        a = diff
        b = start - diff * l
        self.add_linear(l, r, a, b)

    def build(self):
        res = [0] * self.n
        c = 0
        s = 0

        if self.mod is None:
            for i in range(self.n):
                c += self.coef[i]
                s += self.cons[i]
                res[i] = c * i + s
        else:
            mod = self.mod
            for i in range(self.n):
                c = (c + self.coef[i]) % mod
                s = (s + self.cons[i]) % mod
                res[i] = (c * i + s) % mod

        return res
```

## 3.8 MargeSortTree.py

```
from array import array
from bisect import bisect_left, bisect_right


class MergeSortTree:
    """構築 O(n log n)、区間内の個数クエリ O(log n)。"""

    def __init__(self, data):
        self.n = len(data)
        self.size = size = 1 << max(0, self.n - 1).bit_length()
        self.tree = tree = [[]] * (2 * size)
        self._left_prefix = links = [None] * size

        tree[size : size + self.n] = [[x] for x in data]
        # 子ノードの要素をソートしてすべて持つように構築
        for i in range(size - 1, 0, -1):
            left = tree[2 * i]
            values = tree[i] = sorted(left + tree[2 * i + 1])
            left_size = len(left)
            # 親の先頭 k 要素に含まれる左の子の要素数を記録する。
            # 同値の要素は、安定ソートと同じく左の子を先に数える。
            prefix = array("I", [0])
            rank = 0
            for value in values:
                if rank < left_size and not value < left[rank]:
                    rank += 1
                prefix.append(rank)
            links[i] = prefix

    def _count(self, l, r, lo, hi, node, height):
        links = self._left_prefix
        width = 1 << height
        mask = width - 1
        # 両端の累積個数を同じループで求め、その差を取る。
        res = hi - lo if r & mask == 0 else 0
        for pos, sign in ((l & mask, -1), (r & mask, 1)):
            v, a, b, bit = node, lo, hi, width
            count = 0
            while pos and a != b:
                bit >>= 1
                link = links[v]
                x, y = link[a], link[b]
                v <<= 1
                if pos & bit:
                    count += y - x
                    a -= x
                    b -= y
                    v += 1
                    pos -= bit
                else:
                    a, b = x, y
            res += sign * count
        return res

    # 区間 [l, r) で値が x 以下の要素数を求める
    def query_leq(self, l, r, x):
        if l >= r:
            return 0
        # 両端の葉を含む最小ノードでだけ二分探索する。
        height = (l ^ (r - 1)).bit_length()
        node = (l + self.size) >> height
        values = self.tree[node]
        hi = bisect_right(values, x)
        if hi == len(values):
            return r - l
        return self._count(l, r, 0, hi, node, height)

    # 区間 [l, r) で値が a 以上 b 未満の要素数を求める
    def query_range(self, l, r, a, b):
        if l >= r:
            return 0
        height = (l ^ (r - 1)).bit_length()
        node = (l + self.size) >> height
        values = self.tree[node]
        lo = bisect_left(values, a)
        hi = bisect_left(values, b)
        if not lo and hi == len(values):
            return r - l
        return self._count(l, r, lo, hi, node, height)
```

## 3.9 Mos_algorithm.py

```
def mos_algorithm(data, queries):
    """
    data : list of int : 元の配列
    queries : list of tuple : クエリのリスト
    クエリは 0-indexed で半開区間 [l,r)
    """
    from math import sqrt, ceil

    n = len(data)
    q = len(queries)
    B = ceil(sqrt(3) * n / (sqrt(2) * sqrt(q)))  # bukect のサイズ (nyaan さんの記事より)
    L, R = 0, 0
    # query の順番を格納する用
    order = [-1] * q

    ########################################
    # 状態を初期化する処理を書く

    ########################################

    def _add(i, ans):
        # i 番目の要素を含めて考えるときへ states を更新
        ########################################

        return ans
        ########################################

    def _delete(i, ans):
        # i 番目の要素を除いて考えるときへ states を更新
        ########################################

        return ans
        ########################################

    # クエリの解答順
    # (ceil(l/B), r*-1^{ceil(l/B)}) の昇順にするのが最速ぽい
    for idx, (l, r) in enumerate(queries):
        order[idx] = ((l // B) << 40) + ((r if (l // B) & 1 else -r) << 20) + idx
    order.sort()

    mask = (1 << 20) - 1
    ret = [-1] * q
    for lri in order:  # クエリに答えていく
        i = lri & mask
        l, r = queries[i]

        # クエリ [l,r) に対して states を更新する
        while R < r:  # r まで伸長
            ans = _add(R, ans)
            R += 1
        while R > r:  # r まで短縮
            R -= 1
            ans = _delete(R, ans)

        while L < l:  # l まで短縮
            ans = _delete(L, ans)
            L += 1
        while L > l:  # l まで伸長
            L -= 1
            ans = _add(L, ans)

        ########################################
        # クエリに答える作業をここで書く
        # ret[i] に i 番目のクエリの答えを格納する
        ret[i] = ans
        ########################################

    return ret
```

## 3.10 PackedTupleHeap.py

```
"""
PackedTupleHeap(arity=2, bits=20, values=None): 整数タプルの辞書順最小ヒープ。
arity: タプルの長さ
values: list[tuple[int, ...]] (例: [(1, 2, 3)])。省略時の None は空。
bits: 第 2 成分以降の範囲を表す int または整数の列。3 要素 (a, b, c) の場合:
  bits=8        → b, c は各 0〜255。
  bits=(8, 12) → b は 0〜255、c は 0〜4095。
先頭の a は指定しない、負数も可。各成分の最大値が M (0 以上) なら、必要なビット数は M.bit_length()。
h = PackedTupleHeap(3, bits=(8, 12), values=[(-1, 2, 3)])
h.push((0, 4, 5))      # 追加。渡すタプルの長さは常に arity
h.top(); h.pop()       # 最小値の参照 / 取り出し (空なら IndexError)
```

```
len(h); bool(h)       # 要素数 / 非空か
while h: print(h.pop())  # 空になるまで取り出す
"""

from heapq import heapify, heappop, heappush


class PackedTupleHeap:
    def __init__(
        s,
        arity: int = 2,
        bits: int | list[int] = 20,
        values: list[tuple[int, ...]] | None = None,
    ) -> None:
        s.bits = (bits,) * (arity - 1) if isinstance(bits, int) else tuple(bits)
        s.q = [s._pack(x) for x in values or ()]
        heapify(s.q)

    def __len__(s):
        return len(s.q)

    def _pack(s, x):
        x = tuple(x)
        v = x[0]
        for a, b in zip(x[1:], s.bits):
            v = (v << b) | a
        return v

    def _unpack(s, v):
        x = []
        for b in reversed(s.bits):
            x.append(v & ((1 << b) - 1))
            v >>= b
        return (v, *reversed(x))

    def top(s):
        return s._unpack(s.q[0])

    def push(s, x):
        heappush(s.q, s._pack(x))

    def pop(s):
        return s._unpack(heappop(s.q))
```

## 3.11 Segtree.py

```
from warnings import warn


class segtree:  # すべて 0-index
    def __init__(s, v, op, e):
        s.n = len(v)
        s.op = op
        s.e = e
        s.z = 1 << (s.n - 1).bit_length()
        d = s.d = [e] * (2 * s.z)
        d[s.z : s.z + s.n] = v
        for i in range(s.z - 1, 0, -1):
            d[i] = op(d[2 * i], d[2 * i + 1])

    # 1 点更新
    def set(s, p, x):
        assert 0 <= p and p < s.n
        d, op = s.d, s.op
        p += s.z
        d[p] = x
        p >>= 1
        while p:
            d[p] = op(d[2 * p], d[2 * p + 1])
            p >>= 1

    # 1 点加算
    def add(s, p, x):
        assert 0 <= p and p < s.n
        d, op = s.d, s.op
        p += s.z
        d[p] += x
        p >>= 1
        while p:
            d[p] = op(d[2 * p], d[2 * p + 1])
            p >>= 1

    # 元の配列の p 番目の要素を取得
    def get(s, p):
        assert 0 <= p and p < s.n
        return s.d[p + s.z]

    # [l,r) の演算結果を返す
    def prod(s, l, r):
        assert 0 <= l and l <= r and r <= s.n
        if l == r:
            warn("l=r in prod(l,r)")
            return s.e
        d, op = s.d, s.op
        sml = smr = s.e
        l += s.z
        r += s.z
        while l < r:
            if l & 1:
                sml = op(sml, d[l])
                l += 1
            if r & 1:
                smr = op(d[r - 1], smr)
            l >>= 1
            r >>= 1
        return op(sml, smr)

    # 全区間の演算結果を返す
    def all_prod(s):
        return s.d[1]

    # check(operate(data[l],data[l+1],...,data[r-1])) = True
    # を満たす最大の r を返す
    def max_right(s, l, check):
        assert 0 <= l and l <= s.n
        assert check(s.e)
        if l == s.n:
            return s.n
        l += s.z
        sm = s.e
        while True:
            while l % 2 == 0:
                l >>= 1
            if not (check(s.op(sm, s.d[l]))):
                while l < s.z:
                    l = 2 * l
                    if check(s.op(sm, s.d[l])):
                        sm = s.op(sm, s.d[l])
                        l += 1
                return l - s.z
            sm = s.op(sm, s.d[l])
            l += 1
            if (l & -l) == l:
                break
        return s.n

    # check(operate(data[l],data[l+1],...,data[r-1])) = True
    # を満たす最小の l を返す
    def min_left(s, r, check):
        assert 0 <= r and r <= s.n
        assert check(s.e)
        if r == 0:
            return 0
        r += s.z
        sm = s.e
        while True:
            r -= 1
            while r > 1 and (r % 2):
                r >>= 1
            if not (check(s.op(s.d[r], sm))):
                while r < s.z:
                    r = 2 * r + 1
                    if check(s.op(s.d[r], sm)):
                        sm = s.op(s.d[r], sm)
                        r -= 1
                return r + 1 - s.z
            sm = s.op(s.d[r], sm)
            if (r & -r) == r:
                break
        return 0

    # print(seg) で葉の値（元の配列）を表示する
    def __str__(s):
        return str([s.get(i) for i in range(s.n)])
```

## 3.12 SortedMultiset.py

```
# 参考: https://github.com/tatyam-prime/SortedSet/blob/main/SortedMultiset.py
import math
from bisect import bisect_left, bisect_right

class SortedMultiset:
    br = 16
    sr = 24

    def __init__(s, a=[]):
        "ソート済みなら O(N)、それ以外は O(N log N)"
        a = sorted(a)
        n = s.size = len(a)
        k = math.ceil(math.sqrt(n / s.br))
        s.a = [a[n * i // k : n * (i + 1) // k] for i in range(k)]

    def __iter__(s):
        for a in s.a: yield from a

    def _position(s, x):
        "x を挿入するバケット、その添字、バケット内の挿入位置を返す。s は空でないこと。"
        for i, a in enumerate(s.a):
            if x <= a[-1]: break
        return (a, i, bisect_left(a, x))

    def __contains__(s, x):
        if s.size == 0: return False
        a, _, i = s._position(x)
        return i != len(a) and a[i] == x

    def count(s, x):
        "x の出現回数を返す。"
        return s.index_right(x) - s.index(x)

    def add(s, x):
        "x を 1 個追加する。/ O(√ N)"
        if s.size == 0:
            s.a = [[x]]
            s.size = 1
            return
        a, b, i = s._position(x)
        a.insert(i, x)
        s.size += 1
        if len(a) > len(s.a) * s.sr:
            mid = len(a) >> 1
            s.a[b:b+1] = [a[:mid], a[mid:]]

    def _pop(s, a, b, i):
        ans = a.pop(i)
        s.size -= 1
        if not a: del s.a[b]
        return ans

    def discard(s, x):
```

```
        "x を 1 個削除し、削除できたら True を返す。/ O(√ N)"
        if s.size == 0: return False
        a, b, i = s._position(x)
        if i == len(a) or a[i] != x: return False
        s._pop(a, b, i)
        return True

    # lt・gt・index_right は整数用。小数では le・ge・index を複製し、以下のコメントに従って変更する。
    def lt(s, x):
        "x 未満の最大の要素を返す。存在しなければ None を返す。"
        return s.le(x - 1)

    def le(s, x):
        "x 以下の最大の要素を返す。存在しなければ None を返す。"
        for a in reversed(s.a):
            if a[0] <= x:  # lt なら <
                return a[bisect_right(a, x) - 1]  # lt なら bisect_left

    def gt(s, x):
        "x より大きい最小の要素を返す。存在しなければ None を返す。"
        return s.ge(x + 1)

    def ge(s, x):
        "x 以上の最小の要素を返す。存在しなければ None を返す。"
        for a in s.a:
            if a[-1] >= x:  # gt なら >
                return a[bisect_left(a, x)]  # gt なら bisect_right

    def __getitem__(s, i):
        "i 番目の要素を返す。"
        if i < 0:
            for a in reversed(s.a):
                i += len(a)
                if i >= 0: return a[i]
        else:
            for a in s.a:
                if i < len(a): return a[i]
                i -= len(a)
        raise IndexError

    def pop(s, i=-1):
        "i 番目の要素を削除して返す。"
        if i < 0:
            for b, a in enumerate(reversed(s.a)):
                i += len(a)
                if i >= 0: return s._pop(a, ~b, i)
        else:
            for b, a in enumerate(s.a):
                if i < len(a): return s._pop(a, b, i)
                i -= len(a)
        raise IndexError

    def index(s, x):
        "x 未満の要素数を返す。"
        ans = 0
        for a in s.a:
            if a[-1] >= x:  # index_right なら >
                return ans + bisect_left(a, x)  # index_right なら bisect_right
            ans += len(a)
        return ans

    def index_right(s, x):
        "x 以下の要素数を返す。"
        return s.index(x + 1)

    def __len__(s):
        return s.size

    def __str__(s):
        return "{" + str(list(s))[1:-1] + "}"
```

## 3.13 SortedSet.py

```
# 参考: https://github.com/tatyam-prime/SortedSet/blob/main/SortedSet.py
import math
from bisect import bisect_left, bisect_right

class SortedSet:
    br = 16
    sr = 24

    def __init__(s, a=[]):
        "ソートして重複を除く。/ O(N log N)"
        a = sorted(set(a))
        n = s.size = len(a)
        k = math.ceil(math.sqrt(n / s.br))
        s.a = [a[n * i // k : n * (i + 1) // k] for i in range(k)]

    def __iter__(s):
        for a in s.a: yield from a

    def _position(s, x):
        "x を挿入するバケット、その添字、バケット内の挿入位置を返す。s は空でないこと。"
        for i, a in enumerate(s.a):
            if x <= a[-1]: break
        return (a, i, bisect_left(a, x))

    def __contains__(s, x):
        if s.size == 0: return False
        a, _, i = s._position(x)
        return i != len(a) and a[i] == x

    def add(s, x):
        "x を追加し、追加できたら True を返す。/ O(√ N)"
        if s.size == 0:
            s.a = [[x]]
            s.size = 1
            return True
        a, b, i = s._position(x)
        if i != len(a) and a[i] == x: return False
        a.insert(i, x)
        s.size += 1
        if len(a) > len(s.a) * s.sr:
            mid = len(a) >> 1
            s.a[b:b+1] = [a[:mid], a[mid:]]
        return True

    def _pop(s, a, b, i):
        ans = a.pop(i)
        s.size -= 1
        if not a: del s.a[b]
        return ans

    def discard(s, x):
        "x を削除し、削除できたら True を返す。/ O(√ N)"
        if s.size == 0: return False
        a, b, i = s._position(x)
        if i == len(a) or a[i] != x: return False
        s._pop(a, b, i)
        return True

    # lt・gt・index_right は整数用。小数では le・ge・index を複製し、以下のコメントに従って変更する。
    def lt(s, x):
        "x 未満の最大の要素を返す。存在しなければ None を返す。"
        return s.le(x - 1)

    def le(s, x):
        "x 以下の最大の要素を返す。存在しなければ None を返す。"
        for a in reversed(s.a):
            if a[0] <= x:  # lt なら <
                return a[bisect_right(a, x) - 1]  # lt なら bisect_left

    def gt(s, x):
        "x より大きい最小の要素を返す。存在しなければ None を返す。"
        return s.ge(x + 1)

    def ge(s, x):
        "x 以上の最小の要素を返す。存在しなければ None を返す。"
        for a in s.a:
            if a[-1] >= x:  # gt なら >
                return a[bisect_left(a, x)]  # gt なら bisect_right

    def __getitem__(s, i):
        "i 番目の要素を返す。"
        if i < 0:
            for a in reversed(s.a):
                i += len(a)
                if i >= 0: return a[i]
        else:
            for a in s.a:
                if i < len(a): return a[i]
                i -= len(a)
        raise IndexError

    def pop(s, i=-1):
        "i 番目の要素を削除して返す。"
        if i < 0:
            for b, a in enumerate(reversed(s.a)):
                i += len(a)
                if i >= 0: return s._pop(a, ~b, i)
        else:
            for b, a in enumerate(s.a):
                if i < len(a): return s._pop(a, b, i)
                i -= len(a)
        raise IndexError

    def index(s, x):
        "x 未満の要素数を返す。"
        ans = 0
        for a in s.a:
            if a[-1] >= x: # index_right なら >
                return ans + bisect_left(a, x)  # index_right なら bisect_right
            ans += len(a)
        return ans

    def index_right(s, x):
        "x 以下の要素数を返す。"
        return s.index(x + 1)

    def __len__(s):
        return s.size

    def __str__(s):
        return "{" + str(list(s))[1:-1] + "}"
```

## 3.14 SparseTable.py

```
class SparseTable:
    def __init__(self, arr, op):
        """
        arr: 配列
        op: 演算 (結合則、冪等則を満たすこと)
        時間計算量: O(n log n) 構築, O(1) クエリ
        空間計算量: O(n log n)
        """
        self.op = op
        self.n = n = len(arr)
        mk = n.bit_length()
        # table[k][i] = i から長さ 2^k の区間に対する演算結果
        self.tab = tab = [arr[:]] if n else []

        for k in range(1, mk):
            half = 1 << (k - 1)
            prev = tab[-1]
            tab.append([op(prev[i], prev[i + half]) for i in range(n - (1 << k) + 1)])
```

```

    def query(self, l, r):
        """
        [l, r) の区間に対する演算結果を返す O(1)
        """
        assert 0 <= l < r <= self.n
        # 2^k <= r - l を満たす最大の k を求める
        k = (r - l).bit_length() - 1
        # [l, l + 2^k) と [r - 2^k, r) の区間に分割して計算
        return self.op(self.tab[k][l], self.tab[k][r - (1 << k)])
```

## 3.15 TopK.py

```
class SumTopK:
    # 長さ N の配列 A の大きい方から K 個の和を高速に管理する
    def __init__(self, N, A, K):
        self.A = A
        self.ok = SortedMultiset([0] * K)  # 使うやつ
        self.ng = SortedMultiset([0] * (N - K))  # 使わないやつ
        self.ans = 0  # 大きい方から K 個の和

    def add(self, y):  # どこかの値が y に変わる
        self.ok.add(y)
        self.ans += y
        yy = self.ok[0]
        self.ans -= yy
        self.ok.discard(yy)
        self.ng.add(yy)

    def DEL(self, y):  # y を削除する
        if y >= self.ok[0]:
            self.ok.discard(y)
            self.ans -= y
            yy = self.ng[-1]
            self.ng.discard(yy)
            self.ok.add(yy)
            self.ans += yy
        else:
            self.ng.discard(y)

    def update(self, x, y):  # A[x] を y に更新し、K 個の和を返す
        self.add(y)
        self.DEL(self.A[x])
        self.A[x] = y
        return self.ans
```

## 3.16 TupleLazySegtree.py

```
class Columns2:
    __slots__ = ("a", "b")

    def __init__(s, n, e):
        s.a = [e[0]] * n
        s.b = [e[1]] * n

    def __getitem__(s, i):
        return s.a[i], s.b[i]

    def __setitem__(s, i, x):
        s.a[i], s.b[i] = x


class Columns3:
    __slots__ = ("a", "b", "c")

    def __init__(s, n, e):
        s.a = [e[0]] * n
        s.b = [e[1]] * n
        s.c = [e[2]] * n

    def __getitem__(s, i):
        return s.a[i], s.b[i], s.c[i]

    def __setitem__(s, i, x):
        s.a[i], s.b[i], s.c[i] = x


# 4 成分以上は同じ形のクラスを用意し、この辞書に追加する。
COLUMN_TYPES = {2: Columns2, 3: Columns3}


class TupleLazySegtree:
    # タプルをそのまま載せるだけでいい
    def __init__(s, v, op, e, mp, comp, ide):
        s.n = len(v)
        s.log = (max(1, s.n) - 1).bit_length()
        z = s.z = 1 << s.log
        s.e = e = tuple(e)
        s.ide = ide = tuple(ide)
        s.op = op
        s.mp = mp
        s.comp = comp
        assert len(e) in COLUMN_TYPES and len(ide) in COLUMN_TYPES
        d = s.d = COLUMN_TYPES[len(e)](2 * z, e)
        s.lz = COLUMN_TYPES[len(ide)](z, ide)
        s.pend = [False] * z
        for i, value in enumerate(v, z):
            d[i] = value
        for i in range(z - 1, 0, -1):
            d[i] = op(d[2 * i], d[2 * i + 1])

    def _push(s, k):
        pend = s.pend
        if not pend[k]:
            return
        d, lz, mp, comp = s.d, s.lz, s.mp, s.comp
        f = lz[k]
        c = 2 * k
        d[c] = mp(f, d[c])
        d[c + 1] = mp(f, d[c + 1])
        if c < s.z:
            lz[c] = comp(f, lz[c]) if pend[c] else f
            lz[c + 1] = comp(f, lz[c + 1]) if pend[c + 1] else f
            pend[c] = pend[c + 1] = True
        pend[k] = False

    def set(s, p, x):
        """p 番目の値をタプル x に更新する。"""
        assert 0 <= p < s.n
        d, op, push = s.d, s.op, s._push
        p += s.z
        for i in range(s.log, 0, -1):
            push(p >> i)
        d[p] = x
        p >>= 1
        while p:
            d[p] = op(d[2 * p], d[2 * p + 1])
            p >>= 1

    def get(s, p):
        assert 0 <= p < s.n
        push = s._push
        p += s.z
        for i in range(s.log, 0, -1):
            push(p >> i)
        return s.d[p]

    def prod(s, l, r):
        assert 0 <= l <= r <= s.n
        if l == r:
            return s.e
        d, op, push = s.d, s.op, s._push
        l += s.z
        r += s.z
        left = (l & -l).bit_length()
        right = (r & -r).bit_length()
        for i in range(s.log, 0, -1):
            if i >= left:
                push(l >> i)
            if i >= right:
                push(r >> i)
        sml = smr = s.e
        while l < r:
            if l & 1:
                sml = op(sml, d[l])
                l += 1
            if r & 1:
                smr = op(d[r - 1], smr)
            l >>= 1
            r >>= 1
        return op(sml, smr)

    def all_prod(s):
        return s.d[1]

    def apply_point(s, p, f):
        assert 0 <= p < s.n
        d, op, push = s.d, s.op, s._push
        p += s.z
        for i in range(s.log, 0, -1):
            push(p >> i)
        d[p] = s.mp(f, d[p])
        p >>= 1
        while p:
            d[p] = op(d[2 * p], d[2 * p + 1])
            p >>= 1

    def apply(s, l, r, f):
        assert 0 <= l <= r <= s.n
        if l == r:
            return
        d, lz, pend = s.d, s.lz, s.pend
        op, mp, comp = s.op, s.mp, s.comp
        push, z = s._push, s.z
        l += z
        r += z
        left = (l & -l).bit_length()
        right = (r & -r).bit_length()
        for i in range(s.log, 0, -1):
            if i >= left:
                push(l >> i)
            if i >= right:
                push(r >> i)
        l2, r2 = l, r
        while l < r:
            if l & 1:
                d[l] = mp(f, d[l])
                if l < z:
                    lz[l] = comp(f, lz[l]) if pend[l] else f
                    pend[l] = True
                l += 1
            if r & 1:
                r -= 1
                d[r] = mp(f, d[r])
                if r < z:
                    lz[r] = comp(f, lz[r]) if pend[r] else f
                    pend[r] = True
            l >>= 1
            r >>= 1
        for i in range(1, s.log + 1):
            if i >= left:
                k = l2 >> i
                d[k] = op(d[2 * k], d[2 * k + 1])
            if i >= right:
                k = r2 >> i
                d[k] = op(d[2 * k], d[2 * k + 1])

    def max_right(s, l, check):
        """check(prod(l, r)) を満たす最大の r。check(e) が真で単調であること。"""
        assert 0 <= l <= s.n
```

```
        assert check(s.e)
        if l == s.n:
            return s.n
        d, op, push = s.d, s.op, s._push
        z = s.z
        l += z
        for i in range(s.log, 0, -1):
            push(l >> i)
        sm = s.e
        while True:
            while l % 2 == 0:
                l >>= 1
            nxt = op(sm, d[l])
            if not check(nxt):
                while l < z:
                    push(l)
                    l *= 2
                    nxt = op(sm, d[l])
                    if check(nxt):
                        sm = nxt
                        l += 1
                return l - z
            sm = nxt
            l += 1
            if (l & -l) == l:
                return s.n

    def min_left(s, r, check):
        """check(prod(l, r)) を満たす最小の l。check(e) が真で単調であること。"""
        assert 0 <= r <= s.n
        assert check(s.e)
        if r == 0:
            return 0
        d, op, push = s.d, s.op, s._push
        z = s.z
        r += z
        for i in range(s.log, 0, -1):
            push((r - 1) >> i)
        sm = s.e
        while True:
            r -= 1
            while r > 1 and r % 2:
                r >>= 1
            nxt = op(d[r], sm)
            if not check(nxt):
                while r < z:
                    push(r)
                    r = 2 * r + 1
                    nxt = op(d[r], sm)
                    if check(nxt):
                        sm = nxt
                        r -= 1
                return r + 1 - z
            sm = nxt
            if (r & -r) == r:
                return 0

    def __str__(s):
        return str([s.get(i) for i in range(s.n)])
```

## 3.17 TupleSegtree.py

```
class Columns2:
    __slots__ = ("a", "b")

    def __init__(s, n, e):
        s.a = [e[0]] * n
        s.b = [e[1]] * n

    def __getitem__(s, i):
        return s.a[i], s.b[i]

    def __setitem__(s, i, x):
        s.a[i], s.b[i] = x


class Columns3:
    __slots__ = ("a", "b", "c")

    def __init__(s, n, e):
        s.a = [e[0]] * n
        s.b = [e[1]] * n
        s.c = [e[2]] * n

    def __getitem__(s, i):
        return s.a[i], s.b[i], s.c[i]

    def __setitem__(s, i, x):
        s.a[i], s.b[i], s.c[i] = x


# 4 成分以上は同じ形のクラスを用意し、この辞書に追加する。
COLUMN_TYPES = {2: Columns2, 3: Columns3}


class TupleSegtree:  # すべて 0-index
    # タプルをそのまま載せるだけでいい
    def __init__(s, v, op, e):
        s.n = len(v)
        z = s.z = 1 << (max(1, s.n) - 1).bit_length()
        s.e = e = tuple(e)
        s.op = op
        assert len(e) in COLUMN_TYPES
        d = s.d = COLUMN_TYPES[len(e)](2 * z, e)
        for i, value in enumerate(v, z):
            d[i] = value
        for i in range(z - 1, 0, -1):
            d[i] = op(d[2 * i], d[2 * i + 1])

    # 1 点更新
    def set(s, p, x):
        assert 0 <= p < s.n
        d, op = s.d, s.op
        p += s.z
        d[p] = x
        p >>= 1
        while p:
            d[p] = op(d[2 * p], d[2 * p + 1])
            p >>= 1

    # 1 点加算（タプルの成分ごとに加算）
    def add(s, p, x):
        assert 0 <= p < s.n
        assert len(x) == len(s.e)
        d, op = s.d, s.op
        p += s.z
        d[p] = tuple(a + b for a, b in zip(d[p], x))
        p >>= 1
        while p:
            d[p] = op(d[2 * p], d[2 * p + 1])
            p >>= 1

    # 元の配列の p 番目の要素を取得
    def get(s, p):
        assert 0 <= p < s.n
        return s.d[p + s.z]

    # [l,r) の演算結果を返す
    def prod(s, l, r):
        assert 0 <= l <= r <= s.n
        if l == r:
            return s.e
        d, op = s.d, s.op
        sml = smr = s.e
        l += s.z
        r += s.z
        while l < r:
            if l & 1:
                sml = op(sml, d[l])
                l += 1
            if r & 1:
                smr = op(d[r - 1], smr)
            l >>= 1
            r >>= 1
        return op(sml, smr)

    # 全区間の演算結果を返す
    def all_prod(s):
        return s.d[1]

    def max_right(s, l, check):
        """check(prod(l, r)) を満たす最大の r。check(e) が真で単調であること。"""
        assert 0 <= l <= s.n
        assert check(s.e)
        if l == s.n:
            return s.n
        d, op, z = s.d, s.op, s.z
        l += z
        sm = s.e
        while True:
            while l % 2 == 0:
                l >>= 1
            nxt = op(sm, d[l])
            if not check(nxt):
                while l < z:
                    l *= 2
                    nxt = op(sm, d[l])
                    if check(nxt):
                        sm = nxt
                        l += 1
                return l - z
            sm = nxt
            l += 1
            if (l & -l) == l:
                return s.n

    def min_left(s, r, check):
        """check(prod(l, r)) を満たす最小の l。check(e) が真で単調であること。"""
        assert 0 <= r <= s.n
        assert check(s.e)
        if r == 0:
            return 0
        d, op, z = s.d, s.op, s.z
        r += z
        sm = s.e
        while True:
            r -= 1
            while r > 1 and r % 2:
                r >>= 1
            nxt = op(d[r], sm)
            if not check(nxt):
                while r < z:
                    r = 2 * r + 1
                    nxt = op(d[r], sm)
                    if check(nxt):
                        sm = nxt
                        r -= 1
                return r + 1 - z
            sm = nxt
            if (r & -r) == r:
                return 0

    # print(seg) で葉の値（元の配列）を表示する
    def __str__(s):
        return str([s.get(i) for i in range(s.n)])
```

## 3.18 TwoWayList.py

```
# 双方向連結リスト https://atcoder.jp/contests/abc344/editorial/9487
# 挿入・削除 O(1)
```

```python
# 要素の重複不可


class TwoWayList:
    def __init__(self, L):
        self.head = -1
        self.tail = pow(10, 18)
        self.L = [self.head] + L + [self.tail]
        self.front, self.back = dict(), dict()
        for i in range(len(self.L) - 1):
            self.back[self.L[i]] = self.L[i + 1]  # x の後ろ
            self.front[self.L[i + 1]] = self.L[i]  # x の前

    def insert_front(self, x, y):  # x の前に y を挿入する
        # 現在 a -> x
        # a -> y -> x
        a = self.front[x]
        self.back[a] = y
        self.front[x] = y
        self.back[y] = x
        self.front[y] = a

    def insert_back(self, x, y):  # x の直後に y を挿入する
        # 現在 x -> b
        # x -> y -> b
        b = self.back[x]
        self.back[x] = y
        self.front[y] = x
        self.back[y] = b
        self.front[b] = y

    def del_x(self, x):  # x を削除する
        a = self.front[x]
        b = self.back[x]
        self.back[a] = b
        self.front[b] = a
        del self.front[x]
        del self.back[x]

    def res_LIST(self):  # リストを返す
        ans = list()
        crr = self.back[self.head]
        while crr != self.tail:
            ans.append(crr)
            crr = self.back[crr]
        return ans
```

# 4 graph

## 4.1 Bellman_ford.py

```python
inf = pow(10, 18)


def bellman_ford(
    Edges: list[tuple[int, int, int]], N: int, s: int
) -> tuple[list[int], bool]:
    """
    Edges : 有向辺（無向のときは両方入れる）
    N : 頂点数
    s : 始点
    返り値 : （最短距離配列，始点から到達できる負閉路の有無）
    """
    dist = [inf] * (N + 1)
    dist[s] = 0
    cnt = 0
    while cnt < N:
        end = True
        for fro, to, cost in Edges:
            if dist[fro] == inf:
                continue
            if dist[to] > dist[fro] + cost:
                dist[to] = dist[fro] + cost
                end = False
        if end:
            break
        cnt += 1

    return dist, cnt == N
```

## 4.2 BipartiteGraph.py

```python
from collections import deque


# 二部グラフなら True と色分けを返す
def is_bipartite_graph(G: list[list[int]], N: int) -> tuple[bool, list[int]]:
    col = [-1] * (N + 1)

    def dfs(v):
        que = deque()
        que.append(v)
        while que:
            v = que.pop()
            for v2 in G[v]:
                if col[v2] != -1:
                    if col[v] == col[v2]:
                        return False
                    continue
                col[v2] = 1 - col[v]
                que.append(v2)
        return True

    for v in range(1, N + 1):
        if col[v] != -1:
            continue
        col[v] = 1
        if dfs(v) == False:
            return False, []

    return True, col
```

## 4.3 EulerTour.py

```python
class SparseTable:
    def __init__(self, arr, op):
        """
        arr: 配列
        op: 演算（結合則、冪等則を満たすこと）
        時間計算量: O(n log n) 構築, O(1) クエリ
        空間計算量: O(n log n)
        """
        self.op = op
        self.n = n = len(arr)
        mk = n.bit_length()
        # table[k][i] = i から長さ 2^k の区間に対する演算結果
        self.table = table = [arr[:]] if n else []

        for k in range(1, mk):
            half_len = 1 << (k - 1)
            prev = table[-1]
            table.append(
                [op(prev[i], prev[i + half_len]) for i in range(n - (1 << k) + 1)]
            )

    def query(self, l, r):
        """
        [l, r) の区間に対する演算結果を返す O(1)
        """
        assert 0 <= l < r <= self.n
        # 2^k <= r - l を満たす最大の k を求める
        k = (r - l).bit_length() - 1
        # [l, l + 2^k) と [r - 2^k, r) の区間に分割して計算
        return self.op(self.table[k][l], self.table[k][r - (1 << k)])


class fenwick_tree:
    n = 1
    data = [0 for i in range(n)]

    # N 要素の配列を 0 で初期化する
    def __init__(self, N):
        self.n = N
        self.data = [0 for i in range(N)]

    # p 番目の要素に x を加算する
    def add(self, p, x):
        assert 0 <= p < self.n, "0<=p<n,p={0},n={1}".format(p, self.n)
        p += 1
        while p <= self.n:
            self.data[p - 1] += x
            p += p & -p

    # [l, r) の和を返す
    def sum(self, l, r):
        assert 0 <= l and l <= r and r <= self.n,
        "0<=l<=r<=n,l={0},r={1},n={2}".format(
            l, r, self.n
        )
        return self.sum0(r) - self.sum0(l)

    # [0, r) の和を返す
    def sum0(self, r):
        s = 0
        data = self.data
        while r > 0:
            s += data[r - 1]
            r -= r & -r
        return s


class EulerTour:
    # N 頂点の木を初期化する
    def __init__(s, N):
        s.N = N
        s.G = [[] for _ in range(N + 1)]
        s._in = [-1] * (N + 1)  # 各頂点の訪問開始時刻
        s._out = [-1] * (N + 1)  # 各頂点の訪問終了時刻
        s.m = 0  # 辺数
        s.ws = []  # ツアー順の辺の重み
        s.es = []  # ツアー順の辺番号
        s.vs = []  # ツアー順の頂点番号
        s.depth = [-1]
        s.base = 1 << N.bit_length()
        s.mask = s.base - 1

    # 辺の重みの和を管理する Fenwick Tree を O(N) で構築する
    def _build_fentree(s):
        n = len(s.ws)
        s.fen = fenwick_tree(n)
        d = s.fen.data = s.ws[:]
        if d:
            d[0] = 0
        for i in range(1, n + 1):
            j = i + (i & -i)
            if j <= n:
                d[j - 1] += d[i - 1]

    # LCA 用の区間最小値を求める SparseTable を構築する
    def _build_lca(s):
        a = [d * s.base + abs(v) for d, v in zip(s.depth, s.vs)]
        s.lca_table = SparseTable(a, min)

    # 無向辺 u-v を重さ w で追加する
    def add_edge(s, u, v, w):
        s.m += 1
```

```
        s.G[u].append((v, w, s.m))
        s.G[v].append((u, w, s.m))

    # root を根としてオイラーツアーとクエリ用のデータ構造を構築する
    def build(s, root):
        s.root = root
        st = [(root, 0, 0)]
        while st:
            v, w, i = st.pop()
            t = len(s.es)
            if v >= 0:
                s._in[v] = s._out[v] = t
                s.vs.append(v)
                s.depth.append(s.depth[-1] + 1)
                for e in s.G[v]:
                    if s._in[e[0]] < 0:
                        st.append((-v, -e[1], -e[2]))
                        st.append(e)
            else:
                v = ~v
                s._out[v] = t
                s.vs.append(~v)
                s.depth.append(s.depth[-1] - 1)
            s.es.append(i)
            s.ws.append(w)
        s.depth.pop(0)
        s._build_fentree()
        s._build_lca()

    # 根から u へのパスのコストの和を返す
    def path_sum(s, u):
        return s.fen.sum0(s._in[u] + 1)

    # u と v の最小共通祖先を O(1) で返す
    def lca(s, u, v):
        l, r = s._in[u], s._in[v]
        if l > r:
            l, r = r, l
            u = v
        if r <= s._out[u]:
            return u
        return s.lca_table.query(l, r + 1) & s.mask

    # パス u-v のコストの和を返す
    def get_distance(s, u, v):
        a = s.lca(u, v)
        return s.path_sum(u) + s.path_sum(v) - 2 * s.path_sum(a)
```

## 4.4 LCA.py

```
# HLD。参考: https://judge.yosupo.jp/submission/40074
class LCA:
    # n: 頂点数、E: 辺集合, root: 木の根
    def __init__(self, n, E, root=0):
        par = self.par = [0] * (n + 1)
        dep = self.dep = [-1] * (n + 1)
        siz = self.siz = [1] * (n + 1)
        jmp = self.jmp = [-1] * (n + 1)
        tin = self.tin = [0] * (n + 1)
        deg = [0] * (n + 1)

        for u, v in E:
            deg[u] += 1
            deg[v] += 1
            par[u] ^= v
            par[v] ^= u
        que = []
        for u in range(n + 1):
            if deg[u] == 1 and u != root:
                que.append(u)
            elif deg[u] == 0:
                par[u] = -1
        for u in que:
            v = par[u]
            siz[v] += siz[u]
            if siz[u] > tin[v]:
                tin[v] = siz[u]
                jmp[v] = u
            par[v] ^= u
            deg[v] -= 1
            if deg[v] == 1 and v != root:
                que.append(v)
        par[root] = -1

        if jmp[root] >= 0:
            siz[jmp[root]] = -siz[jmp[root]]
        jmp[root] = -1
        siz[root] = tin[root] + 1
        dep[root] = tin[root] = 0
        for u in reversed(que):
            v = par[u]
            dep[u] = dep[v] + 1
            h, b = jmp[u], tin[u] + 1
            if h >= 0:
                siz[h] = -siz[h]
            if siz[u] < 0:
                tin[u] = tin[v] + 1
                jmp[u] = jmp[v]
            else:
                tin[u] = tin[v] + siz[v]
                siz[v] += siz[u]
                jmp[u] = v
            siz[u] = b

    # u と v の LCA を返す O(log N)
    def lca(self, u, v):
        tin, jmp = self.tin, self.jmp
        l, r = tin[u], tin[v]
        if l > r:
            l, r = r, l
            u, v = v, u
        if r < l + self.siz[u]:
            return u
        v = jmp[v]
        while l < tin[v]:
            v = jmp[v]
        return v

    # 構築時の根を基準に、u の部分木に v が含まれるか O(1)
    def is_in_subtree(self, u, v):
        tin = self.tin
        return tin[u] <= tin[v] < tin[u] + self.siz[u]

    # u と v の距離を返す
    def get_distance(self, u, v):
        dep = self.dep
        return dep[u] + dep[v] - 2 * dep[self.lca(u, v)]

    # u-v のパスが via を通るか（端点も含む）
    def is_on_path(self, u, v, via):
        anc_u = self.is_in_subtree(via, u)
        anc_v = self.is_in_subtree(via, v)
        return anc_u != anc_v or anc_u and self.lca(u, v) == via
```

## 4.5 PotentialUnionFind.py

```
class PotentialUnionFind:
    def __init__(self, N) -> None:
        self.N = N
        self.parents = [-1] * (N + 1)
        # 各要素の根からの距離
        # 相対的な重さを表す
        # 経路圧縮前は親ノードとの差分
        self.potential = [0] * (N + 1)
        self.inf = pow(10, 18)
        self.inconsistent = [False] * (
            N + 1
        )  # 連結成分に負閉路があるか（矛盾があるか）

    # 頂点 x の根を返す関数
    # O(α (N))
    def root(self, x):
        if self.parents[x] < 0:
            return x
        else:
            # 経路圧縮
            r = self.root(self.parents[x])  # 先に経路圧縮してポテンシャルを更新する
            self.potential[x] += self.potential[
                self.parents[x]
            ]  # ポテンシャルは根までの合計（根につないだ時につじつま合うように）
            self.parents[x] = r
            return self.parents[x]

    # potential[y] - potential[x]
    # y が x よりどれだけ重いか
    # 連結でないなら inf を返す
    # O(α (N))
    def dist(self, x, y):
        rx = self.root(x)
        ry = self.root(y)
        if rx == ry:  # 同じ連結成分に属するなら
            return self.potential[y] - self.potential[x]
        else:
            return self.inf

    # x と y を距離 distance でつなぐ
    # 正確には potential[y] = potential[x] + distance になるようにつなぐ
    # x の下に y をつなぐ
    # y は x より w 重い
    # x -> y の重み distance の有向辺
    # 返り値は矛盾がなければ true
    # O(α (N))
    def union(self, x, y, w):
        rx = self.root(x)
        ry = self.root(y)
        if rx == ry:  # 既に連結なら
            if self.potential[y] - self.potential[x] != w:
                self.inconsistent[rx] = True
            return self.potential[y] - self.potential[x] == w
        # ややこしいが、x-y をつなぐが、経路圧縮の都合で実装では rx-ry をつなぐことになる
        # union by size でつなげる
        if self.parents[ry] > self.parents[rx]:
            # rx の下に ry をつなぐ
            self.parents[rx] += self.parents[ry]
            self.potential[ry] = self.potential[x] + w - self.potential[y]
            self.parents[ry] = rx
        else:
            # ry の下に rx をつなぐ
            self.parents[ry] += self.parents[rx]
            self.potential[rx] = self.potential[y] - w - self.potential[x]
            self.parents[rx] = ry
        return True

    # 要素 x が属するグループの要素数を返す
    # O(α (N))
    def size(self, x):
        return -self.parents[self.root(x)]

    # x,y が同じグループか（連結か）を返す関数
    # O(α (N))
    def isSame(self, x, y):
        return self.root(x) == self.root(y)

    # x が属するグループの要素をリストで返す
    # O(N)
    def members(self, x):
        root = self.root(x)
        return [i for i in range(1, self.N + 1) if self.root(i) == root]

```

```
    # すべての根の要素をリストで返す
    # O(N × α (N))
    def roots(self):
        return [i for i, x in enumerate(self.parents) if x < 0]

    # グループの数を返す
    # O(N × α (N))
    def group_count(self):
        return len(self.roots())
```

## 4.6 Rerooting.py

```
# 全方位木 DP
class rerooting:
    def __init__(
        self,
        n: int,
        edges: list[tuple[int, int]],
        identity: int,
        merge: callable,
        calc_node_value: callable,
        calc_ans: callable,
    ):
        self.n = n
        self.identity = identity
        self.merge = merge
        self.calc_node_value = calc_node_value
        self.calc_ans = calc_ans
        # adjacents[i] = i と隣接する頂点のリスト
        self.adjacents = [[] for _ in range(n + 1)]
        # indexForAdjacents[i][j] = adjacents[i][j](i の j 番目の隣接頂点) にとって、i が何番目の隣接頂点か
        # adjacents[adjacents[i][j]][indexForAdjacents[i][j]] == i である
        self.indexForAdjacents = [[] for _ in range(n + 1)]
        for a, b in edges:
            self.indexForAdjacents[a].append(len(self.adjacents[b]))
            self.indexForAdjacents[b].append(len(self.adjacents[a]))
            self.adjacents[a].append(b)
            self.adjacents[b].append(a)
        # childSubTreeValue[i] = i の子の部分木の値
        # childSubTreeValue[i][j] = i を親とし、adjacents[i][j] を根とした部分木の値
        self.childSubTreeValue = [
            [self.identity] * len(self.adjacents[i]) for i in range(n + 1)
        ]
        # ans[i] = i を根としたときの求めたい値
        self.ans = [self.identity] * (n + 1)

    # root から DFS して、子の部分木の値を計算
    def _dfs1(self, root):
        self.parent = [-1] * (self.n + 1)
        self.order = [-1] * self.n  # DFS での訪問順
        stack = [root]
        idx = 0
        while stack:
            v = stack.pop()
            self.order[idx] = v
            idx += 1
            for u in self.adjacents[v]:
                if u == self.parent[v]:
                    continue
                self.parent[u] = v
                stack.append(u)
        # 訪問順を逆順にして、子から値を計算していく
        for v in reversed(self.order[1:]):
            pv = self.parent[v]
            result = self.identity
            parentIndex = -1
            for i, u in enumerate(self.adjacents[v]):
                if u == pv:
                    parentIndex = i
                    continue
                result = self.merge(result, self.childSubTreeValue[v][i])
            # v を根とした部分木の値を親側に記録
            self.childSubTreeValue[pv][
                self.indexForAdjacents[v][parentIndex]
            ] = self.calc_node_value(result, v)

    # 親の部分木の値を使って、自分を根とした部分木の値を計算
    def _dfs2(self):
        for v in self.order:
            # 子の部分木の値の左からの累積
            accFromLeft = self.identity
            # 子の部分木の値の右からの累積
            # accFromRight[i] = merge(childSubTreeValue[v][i+1], childSubTreeValue[v][i+2], ..., childSubTreeValue[v][len(adjacents[v])-1])
            accFromRight = [self.identity] * len(self.adjacents[v])
            for i in reversed(range(len(self.adjacents[v]) - 1)):
                accFromRight[i] = self.merge(
                    self.childSubTreeValue[v][i + 1], accFromRight[i + 1]
                )

            for j in range(len(self.adjacents[v])):
                u = self.adjacents[v][j]
                # u が親、v が根の部分木の値を計算 (u を考慮しないときの値)
                # maerge(accFromLeft, accFromRight[j]) で u 以外の v の子の値を計算
                result = self.calc_node_value(
                    self.merge(accFromLeft, accFromRight[j]), v
                )
                # いま親とした頂点の配列に記録
                self.childSubTreeValue[u][self.indexForAdjacents[v][j]] = result
                # accFromLeft を更新
                accFromLeft = self.merge(accFromLeft, self.childSubTreeValue[v][j])
            # v を根としたときの求めたい値を計算
            self.ans[v] = self.calc_ans(accFromLeft, v)

    def run(self, root=1):
        self._dfs1(root)
        self._dfs2()
        return self.ans


"""
identity = 頂点に持たせる値 (関数の a に渡される) の単位元
merge(a, b) = 部分木の値 a, b をマージする関数
calc_node_value(a, v) = すべての子部分木の値をマージした値 a から頂点 v を根としたときの計算に使う値 (DP の値) を計算する関数
calc_ans(a, v) = すべての子部分木の値をマージした値 a から頂点 v を根としたときの求めたいものを計算する関数
"""

# identity =
# def merge(a,b):
#   return
# def calc_node_value(a,v):
#   return
# def calc_ans(a,v):
#     return


# rr = rerooting(N, edges, identity, merge, calc_node_value, calc_ans)
```

## 4.7 SCC.py

```
class SCC:
    def __init__(self, N: int):
        self.N = N  # 頂点数
        self.edges = []  # 辺

    def addEdge(self, frm: int, to: int):
        assert 0 <= frm <= self.N
        assert 0 <= to <= self.N
        self.edges.append([frm, to])

    # 辺を CSR 形式にする (別に CSR じゃなくてもできるが高速化のため)
    def _csr(self, N: int, edges: list[list]):
        start = [0] * (N + 1)
        # dst := 辺を始点の昇順にソートした時の終点のリスト
        dst = [0] * len(edges)

        # start[i+1] := 頂点 i を始点とする辺の数 となるように構築
        for u, _ in edges:  # u を始点とする辺
            start[u + 1] += 1
        # start[i] := 始点の頂点番号が i 未満である辺の数 になるように累積和を取る
        # このとき、start[i] = dst で頂点 i が始点となる最小の index (0-index)
        for i in range(1, N + 1):
            start[i] += start[i - 1]

        # pos[i] := 始点を i とする辺の dst への挿入位置
        pos = start[:]
        for u, v in edges:
            dst[pos[u]] = v
            pos[u] += 1

        return start, dst

    # 強連結成分分解をする
    def _scc_ids(self):
        N = self.N
        start, dst = self._csr(N, self.edges)
        it = start[:-1]  # 各頂点で次に調べる辺の位置
        timer = 0  # 訪れた頂点数
        count = 0  # 見つかった強連結成分の数
        active = []  # 探索済みかつ所属成分が未確定の頂点
        lowlink = [0] * N
        order = [-1] * N  # 頂点の行きがけ順
        parent = [-1] * N
        comp = [0] * N

        for root in range(N):
            if order[root] != -1:
                continue
            stack = [root]
            lowlink[root] = order[root] = timer
            timer += 1
            active.append(root)
            while stack:
                v = stack[-1]
                i = it[v]
                if i < start[v + 1]:
                    it[v] = i + 1
                    to = dst[i]
                    if order[to] == -1:
                        parent[to] = v
                        lowlink[to] = order[to] = timer
                        timer += 1
                        active.append(to)
                        stack.append(to)
                    elif lowlink[v] > order[to]:
                        lowlink[v] = order[to]
                else:  # 帰りがけ
                    stack.pop()
                    if lowlink[v] == order[v]:
                        while True:
                            u = active.pop()
                            order[u] = N  # 確定した成分を lowlink の計算から除く
                            comp[u] = count
                            if u == v:
                                break
                        count += 1
                    pv = parent[v]
                    if pv != -1 and lowlink[pv] > lowlink[v]:
                        lowlink[pv] = lowlink[v]
        # 縮約後のグラフで comp をトポロジカルソート順にする
        for v in range(N):
            comp[v] = count - 1 - comp[v]

```

```
        return count, comp

    # groups[i] = グループ ID i に属する頂点の集合を 返す
    def build_scc(self) -> list[list[int]]:
        count, comp = self._scc_ids()
        groups = [[] for _ in range(count)]
        for v in range(self.N):
            groups[comp[v]].append(v)
        return groups

    # 強連結成分分解後のグラフを構築する (同一の強連結成分内の頂点は縮約)
    def build_dag(self) -> list[list[int]]:
        # comp[i] = 頂点 i が属する強連結成分の ID
        count, comp = self._scc_ids()
        # dag[i] = 強連結成分 i から出る辺の行き先の強連結成分の ID
        dag = [[] for _ in range(count)]
        for u, v in self.edges:
            if comp[u] != comp[v]:
                dag[comp[u]].append(comp[v])
        return dag
```

## 4.8 TopologicalSort.py

```
def TopologicalSort(N, G):
    indegree = [0] * (N + 1)
    for v in range(1, N + 1):
        for v2 in G[v]:
            indegree[v2] += 1

    que = [v for v in range(1, N + 1) if indegree[v] == 0]

    # 入次数 0 の頂点とその頂点から出る辺を削除する
    for v in que:
        for v2 in G[v]:
            indegree[v2] -= 1
            if indegree[v2] == 0:
                que.append(v2)

    # 閉路があるなら空の配列を返す
    if len(que) != N:
        return []
    else:
        return que
```

## 4.9 UnionFind.py

```
from collections import defaultdict


class UnionFind:
    def __init__(self, n):
        self.n = n
        # 各頂点の親 (1 個上の要素) を記入
        # その頂点が根なら -1 以下の値
        # parents*-1 がその木のサイズ (要素数)
        self.parents = [-1] * n

    # 頂点 x の根を返す関数
    # 根まで上にたどる
    # 経路圧縮で x から根までに通る頂点をすべて根につなぎなおす
    # 経路圧縮によってこの関数が返す値は変わらない (根を返すから)
    # O(α (N))
    def root(self, x):
        if self.parents[x] < 0:
            return x  # x が根なら x を返す
        else:
            self.parents[x] = self.root(self.parents[x])  # 経路圧縮
            return self.parents[x]

    # マージ後の根を返す O(α (N))
    def union(self, x, y):
        x = self.root(x)
        y = self.root(y)
        if x == y:
            return x  # x,y が同じグループならなにもしない
        # x,y は根であり、parents の値*-1 がその木のサイズ (要素数)
        # サイズが小さい方につなげる (union by size)
        if self.parents[x] > self.parents[y]:
            x, y = y, x  # x が小さい方になるようにする
        self.parents[x] += self.parents[y]  # サイズを更新
        self.parents[y] = x
        return x

    # 要素 x が属するグループの要素数を返す O(α (N))
    def size(self, x):
        return -self.parents[self.root(x)]

    # x,y が同じグループか (連結か) を返す関数 O(α (N))
    def isSame(self, x, y):
        return self.root(x) == self.root(y)

    # x が属するグループの要素をリストで返す O(N)
    def members(self, x):
        root = self.root(x)
        return [i for i in range(self.n) if self.root(i) == root]

    # すべての根の要素をリストで返す O(N)
    def roots(self):
        return [i for i, x in enumerate(self.parents) if x < 0]

    # グループの数を返す O(N)
    def group_count(self):
        return len(self.roots())

    # すべてのグループをリストで返す O(N)
    def all_group_members(self):
        group_members = defaultdict(list)
        for i in range(self.n):
            root = self.root(i)
            group_members[root].append(i)
        return list(group_members.values())
```

## 4.10 WarshallFloyd.py

```
# タイプ数削減、余計なことを考えなくていいように、inf,N,G はグローバル変数
# 無向グラフ、1-index


def WarshallFloyd():
    dist = [[inf] * (N + 1) for _ in range(N + 1)]
    for i in range(N + 1):
        dist[i][i] = 0
    for i in range(1, N + 1):
        for j, c in G[i]:
            if dist[i][j] > c:
                dist[i][j] = c
            if dist[j][i] > c:
                dist[j][i] = c

    for k in range(1, N + 1):
        for i in range(1, N + 1):
            for j in range(1, N + 1):
                if i == j:
                    continue
                if dist[i][j] > dist[i][k] + dist[k][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]

    return dist
```

## 4.11 cycle_detect.py

```
vis = [False] * (N + 1)
fin = [False] * (N + 1)
history = deque()


# 頂点 v の連結成分内の閉路を返す
def cycle_detect(v):
    sta = deque()
    sta.append((v, 0))
    cycle_start = -1
    while sta and cycle_start == -1:
        v, prev = sta.pop()
        if prev == -1:
            history.pop()
            fin[v] = True
            continue
        if vis[v]:
            continue
        vis[v] = True
        history.append(v)
        for v2 in G[v]:
            if v2 == prev:
                continue
            if fin[v2]:
                continue
            if vis[v2]:  # 閉路検出
                cycle_start = v2
                break
            sta.append((v2, -1))
            sta.append((v2, v))

    cycle = list()
    if cycle_start == -1:
        return cycle
    while history:
        v = history.pop()
        cycle.append(v)
        if v == cycle_start:
            break
    return cycle
```

## 4.12 maxflow.py

```
# Dinic 法 O(V^2 E)
class maxflow_edge:
    def __init__(self, to, cap, rev_idx):
        self.to = to  # 行き先
        self.cap = cap  # 容量
        self.rev_idx = rev_idx  # 逆向きの辺のインデックス


class maximum_flow:
    def __init__(self, N):
        self.G = [[] for _ in range(N + 1)]
        self.N = N

    def add_edge(self, a, b, c):
        self.G[a].append(maxflow_edge(b, c, len(self.G[b])))
        self.G[b].append(maxflow_edge(a, 0, len(self.G[a]) - 1))  # 逆向き

    def dfs(self, v, goal, f):
        graph, level, it = self.G, self.level, self.it
        vertices = [v]
        path = []
        limits = [f]
        while vertices:
            v = vertices[-1]
            if v == goal:
                flow = limits[-1]
                for edge in path:
                    edge.cap -= flow
                    graph[edge.to][edge.rev_idx].cap += flow
                return flow

```

```
            row = graph[v]
            i = it[v]
            while i < len(row):
                edge = row[i]
                if edge.cap > 0 and level[edge.to] == level[v] + 1:
                    it[v] = i
                    vertices.append(edge.to)
                    path.append(edge)
                    limits.append(min(limits[-1], edge.cap))
                    break
                i += 1
            else:
                it[v] = i
                level[v] = -1  # この頂点からはこれ以上流せない
                vertices.pop()
                limits.pop()
                if path:
                    path.pop()
                    it[vertices[-1]] += 1
        return 0

    def maxflow(self, start, goal):
        assert start != goal
        graph = self.G
        total_flow = 0
        inf = sum(edge.cap for edge in graph[start])
        while True:
            level = self.level = [-1] * (self.N + 1)
            level[start] = 0
            que = [start]
            for v in que:
                for edge in graph[v]:
                    if edge.cap > 0 and level[edge.to] == -1:
                        level[edge.to] = level[v] + 1
                        que.append(edge.to)
            if level[goal] == -1:
                return total_flow

            self.it = [0] * (self.N + 1)
            while True:
                res = self.dfs(start, goal, inf)
                if res == 0:
                    break
                total_flow += res
```

# 5 hash

## 5.1 MultisetHash.py

```
from random import randint


class MultisetHash:
    # 比較したい配列を引数に渡す
    def __init__(self, A: list, B: list) -> None:
        self.mod = (1 << 61) - 1
        self.hash = dict()
        for a in A:
            self.hash[a] = randint(1, self.mod - 1)
        for b in B:
            self.hash[b] = randint(1, self.mod - 1)
        # ハッシュを取る
        self.hashA, self.hashB = [0], [0]
        for a in A:
            self.hashA.append((self.hashA[-1] + self.hash[a]) % self.mod)
        for b in B:
            self.hashB.append((self.hashB[-1] + self.hash[b]) % self.mod)

    # 1-index(A_1, A_2, ...)
    # A[al:ar+1)==B[bl:br+1) を返す
    def isEqual(self, al, ar, bl, br):
        return (self.hashA[ar] - self.hashA[al - 1]) % self.mod == (
            self.hashB[br] - self.hashB[bl - 1]
        ) % self.mod
```

## 5.2 ZobristHash.py

```
from random import randint


class ZobristHash:
    # 比較したい配列を引数に渡す
    def __init__(self, A: list, B: list) -> None:
        MAX = (1 << 61) - 1
        self.hash = dict()
        for a in A:
            self.hash[a] = randint(1, MAX)
        for b in B:
            self.hash[b] = randint(1, MAX)
        # ハッシュを取る
        self.hashA, self.hashB = [0], [0]
        setA, setB = set(), set()
        for a in A:
            if a in setA:
                self.hashA.append(self.hashA[-1])
            else:
                self.hashA.append(self.hashA[-1] ^ self.hash[a])
                setA.add(a)
        for b in B:
            if b in setB:
                self.hashB.append(self.hashB[-1])
            else:
                self.hashB.append(self.hashB[-1] ^ self.hash[b])
                setB.add(b)

    # 1-index(A_1, A_2, ...)
    # (A_al,A_{al+1},...,A_ar) == (B_al,B_{al+1},...,B_ar) を返す
    def isEqual(self, al, ar, bl, br):
        return (self.hashA[ar] - self.hashA[al - 1]) == (
            self.hashB[br] - self.hashB[bl - 1]
        )
```

## 5.3 tupleToHash.py

```
# 10^6 くらいまでは 22 で大丈夫
# 22 で 3 つとかならいける
# タイプ数を減らすためにグローバル変数

SIZE = 3
LEN = 0
MASK = (1 << 22) - 1


def to_hash(x, y, z):
    res = 0
    res |= x << 44
    res |= y << 22
    res |= z
    return res


def rev(h):
    x = (h >> 44) & mask
    y = (h >> 22) & mask
    z = h & mask
    return x, y, z
```

# 6 math

## 6.1 AngularSort.py

```
def compare(a: tuple[int, int], b: tuple[int, int]):
    ax, ay = a
    bx, by = b

    # 領域判定 (0: 上半平面 [0, 180 度), 1: 下半平面 [180, 360 度))
    # ay < 0 または (ay == 0 かつ ax < 0) の場合が下半平面
    ah = 1 if (ay < 0 or (ay == 0 and ax < 0)) else 0
    bh = 1 if (by < 0 or (by == 0 and bx < 0)) else 0
    # 領域が異なる場合上半平面を優先
    if ah != bh:
        return -1 if ah < bh else 1

    # 外積計算 (ax*by - ay*bx)
    # 正なら反時計回り (a が先)、負なら時計回り (b が先)
    cross = ax * by - ay * bx
    if cross > 0:
        return -1
    if cross < 0:
        return 1
    return 0


def is_same_angle(p1, p2):
    """
    2 点 p1, p2 が原点から見て同じ偏角（同じ方向）にあるか判定する。
    ※原点 (0,0) が含まれる場合は False
    """
    ax, ay = p1
    bx, by = p2

    # 1. 外積が 0 でなければ、そもそも同一直線上にない
    if ax * by - ay * bx != 0:
        return False

    # 2. 内積が正でなければ、逆向き（180 度反対）または原点を含んでいる
    #    内積 > 0 ならば同じ向き
    if ax * bx + ay * by > 0:
        return True

    return False


# 使用例
# from algorithm_library.python.techniques.CmpSort import cmp_sort
# A = [(0,1),(1,1),(1,0),(1,-1),(0,-1),(-1,-1),(-1,0),(-1,1)]
# A = cmp_sort(A, compare)
# ※ compare で偏角を並べる際は原点 (0, 0) を別に扱う。
```

## 6.2 FFT.py

```
"""
shakayami 製 ACL をそのまま窃盗
https://github.com/shakayami/ACL-for-python/blob/master/convolution.py
本家 ACL ドキュメント :
  ↪ https://github.com/atcoder/ac-library/blob/master/document_ja/convolution.md
"""


class FFT:
    def primitive_root_constexpr(self, m):
        if m == 2:
            return 1
        if m == 167772161:
            return 3
        if m == 469762049:
            return 3
        if m == 754974721:
            return 11
        if m == 998244353:
            return 3
        divs = [0] * 20
```

```
        divs[0] = 2
        cnt = 1
        x = (m - 1) // 2
        while x % 2 == 0:
            x //= 2
        i = 3
        while i * i <= x:
            if x % i == 0:
                divs[cnt] = i
                cnt += 1
                while x % i == 0:
                    x //= i
            i += 2
        if x > 1:
            divs[cnt] = x
            cnt += 1
        g = 2
        while 1:
            ok = True
            for i in range(cnt):
                if pow(g, (m - 1) // divs[i], m) == 1:
                    ok = False
                    break
            if ok:
                return g
            g += 1

    def bsf(self, x):
        res = 0
        while x % 2 == 0:
            res += 1
            x //= 2
        return res

    rank2 = 0
    root = []
    iroot = []
    rate2 = []
    irate2 = []
    rate3 = []
    irate3 = []

    def __init__(self, MOD):
        self.mod = MOD
        self.g = self.primitive_root_constexpr(self.mod)
        self.rank2 = self.bsf(self.mod - 1)
        self.root = [0 for i in range(self.rank2 + 1)]
        self.iroot = [0 for i in range(self.rank2 + 1)]
        self.rate2 = [0 for i in range(self.rank2)]
        self.irate2 = [0 for i in range(self.rank2)]
        self.rate3 = [0 for i in range(self.rank2 - 1)]
        self.irate3 = [0 for i in range(self.rank2 - 1)]
        self.root[self.rank2] = pow(self.g, (self.mod - 1) >> self.rank2,
        ↪  self.mod)
        self.iroot[self.rank2] = pow(self.root[self.rank2], self.mod - 2,
        ↪  self.mod)
        for i in range(self.rank2 - 1, -1, -1):
            self.root[i] = (self.root[i + 1] ** 2) % self.mod
            self.iroot[i] = (self.iroot[i + 1] ** 2) % self.mod
        prod = 1
        iprod = 1
        for i in range(self.rank2 - 1):
            self.rate2[i] = (self.root[i + 2] * prod) % self.mod
            self.irate2[i] = (self.iroot[i + 2] * iprod) % self.mod
            prod = (prod * self.iroot[i + 2]) % self.mod
            iprod = (iprod * self.root[i + 2]) % self.mod
        prod = 1
        iprod = 1
        for i in range(self.rank2 - 2):
            self.rate3[i] = (self.root[i + 3] * prod) % self.mod
            self.irate3[i] = (self.iroot[i + 3] * iprod) % self.mod
            prod = (prod * self.iroot[i + 3]) % self.mod
            iprod = (iprod * self.root[i + 3]) % self.mod

    def butterfly(self, a):
        n = len(a)
        h = (n - 1).bit_length()

        LEN = 0
        while LEN < h:
            if h - LEN == 1:
                p = 1 << (h - LEN - 1)
                rot = 1
                for s in range(1 << LEN):
                    offset = s << (h - LEN)
                    for i in range(p):
                        l = a[i + offset]
                        r = a[i + offset + p] * rot
                        a[i + offset] = (l + r) % self.mod
                        a[i + offset + p] = (l - r) % self.mod
                    rot *= self.rate2[(~s & -~s).bit_length() - 1]
                    rot %= self.mod
                LEN += 1
            else:
                p = 1 << (h - LEN - 2)
                rot = 1
                imag = self.root[2]
                for s in range(1 << LEN):
                    rot2 = (rot * rot) % self.mod
                    rot3 = (rot2 * rot) % self.mod
                    offset = s << (h - LEN)
                    for i in range(p):
                        a0 = a[i + offset]
                        a1 = a[i + offset + p] * rot
                        a2 = a[i + offset + 2 * p] * rot2
                        a3 = a[i + offset + 3 * p] * rot3
                        a1na3imag = (a1 - a3) % self.mod * imag
                        a[i + offset] = (a0 + a2 + a1 + a3) % self.mod
                        a[i + offset + p] = (a0 + a2 - a1 - a3) % self.mod
                        a[i + offset + 2 * p] = (a0 - a2 + a1na3imag) % self.mod
                        a[i + offset + 3 * p] = (a0 - a2 - a1na3imag) % self.mod
                    rot *= self.rate3[(~s & -~s).bit_length() - 1]
                    rot %= self.mod
                LEN += 2

    def butterfly_inv(self, a):
        n = len(a)
        h = (n - 1).bit_length()
        LEN = h
        while LEN:
            if LEN == 1:
                p = 1 << (h - LEN)
                irot = 1
                for s in range(1 << (LEN - 1)):
                    offset = s << (h - LEN + 1)
                    for i in range(p):
                        l = a[i + offset]
                        r = a[i + offset + p]
                        a[i + offset] = (l + r) % self.mod
                        a[i + offset + p] = (l - r) * irot % self.mod
                    irot *= self.irate2[(~s & -~s).bit_length() - 1]
                    irot %= self.mod
                LEN -= 1
            else:
                p = 1 << (h - LEN)
                irot = 1
                iimag = self.iroot[2]
                for s in range(1 << (LEN - 2)):
                    irot2 = (irot * irot) % self.mod
                    irot3 = (irot * irot2) % self.mod
                    offset = s << (h - LEN + 2)
                    for i in range(p):
                        a0 = a[i + offset]
                        a1 = a[i + offset + p]
                        a2 = a[i + offset + 2 * p]
                        a3 = a[i + offset + 3 * p]
                        a2na3iimag = (a2 - a3) * iimag % self.mod
                        a[i + offset] = (a0 + a1 + a2 + a3) % self.mod
                        a[i + offset + p] = (a0 - a1 + a2na3iimag) * irot %
                        ↪  self.mod
                        a[i + offset + 2 * p] = (a0 + a1 - a2 - a3) * irot2 %
                        ↪  self.mod
                        a[i + offset + 3 * p] = (
                            (a0 - a1 - a2na3iimag) * irot3 % self.mod
                        )
                    irot *= self.irate3[(~s & -~s).bit_length() - 1]
                    irot %= self.mod
                LEN -= 2

    def convolution(self, a: list[int], b: list[int]) -> list[int]:
        """
        res[i] = Σ_{j=0}^{i} a[j]b[i-j] (0-indexed)
        わかりやすくいうと、インデックスの和が i になるような a, b の積の総和
        ex. a = [1, 2, 3], b = [4, 5, 6]
        res = [1*4, 1*5 + 2*4, 1*6 + 2*5 + 3*4, 2*6 + 3*5, 3*6]
        = [4, 13, 28, 27, 18]
        (インデックスでいうと、res[0] = a[0]*b[0], res[1] = a[0]*b[1] + a[1]*b[0],
    ↪  ...)

        時間計算量
        L = |a| + |b| として、O(L log L + log mod)
        """
        n = len(a)
        m = len(b)
        if not (a) or not (b):
            return []
        if min(n, m) <= 40:
            res = [0] * (n + m - 1)
            for i in range(n):
                for j in range(m):
                    res[i + j] += a[i] * b[j]
                    res[i + j] %= self.mod
            return res
        z = 1 << ((n + m - 2).bit_length())
        a = a + [0] * (z - n)
        b = b + [0] * (z - m)
        self.butterfly(a)
        self.butterfly(b)
        c = [(a[i] * b[i]) % self.mod for i in range(z)]
        self.butterfly_inv(c)
        iz = pow(z, self.mod - 2, self.mod)
        for i in range(n + m - 1):
            c[i] = (c[i] * iz) % self.mod
        return c[: n + m - 1]
```

## 6.3 MatrixPower.py

```
def vec_mul(v: list[int], M: list[list[int]], mod: int):
    n = len(v)
    assert n == len(M)
    res = [0] * n
    for i in range(n):
        x, row = v[i], M[i]
        for j in range(n):
            res[j] = (res[j] + x * row[j]) % mod
    return res


# 行列 A と 行列 B の積を返す
def mat_mul(A: list[list[int]], B: list[list[int]], mod):
    assert len(A[0]) == len(B)
    K = len(B)
    end = K & -8
    BT = list(zip(*B))
    res = []
    for row in A:
        out = []
        for col in BT:
            total = 0
            # 多倍長整数の成長を抑えるため、8 項ごとに剰余を取る
            for k in range(0, end, 8):
                total = (
```

```
                    total
                    + row[k] * col[k]
                    + row[k + 1] * col[k + 1]
                    + row[k + 2] * col[k + 2]
                    + row[k + 3] * col[k + 3]
                    + row[k + 4] * col[k + 4]
                    + row[k + 5] * col[k + 5]
                    + row[k + 6] * col[k + 6]
                    + row[k + 7] * col[k + 7]
                ) % mod
            for k in range(end, K):
                total += row[k] * col[k]
            out.append(total % mod)
        res.append(out)
    return res


# 行列累乗 M^K
def mat_pow(M: list[list[int]], K: int, mod: int):
    res = []
    while K > 0:
        if K & 1:
            if not res:
                # 最初の単位行列との積は、剰余を取ったコピーで置き換える
                res = [[x % mod for x in row] for row in M]
            else:
                res = mat_mul(M, res, mod)
        K >>= 1
        if K:
            M = mat_mul(M, M, mod)

    if not res:
        res = [[0] * len(M) for _ in range(len(M))]
        for i in range(len(M)):
            res[i][i] = 1
    return res
```

## 6.4 PIE.py

```
# 約数系包除に使うやつ系


class eratosthenes:
    def __init__(self, MAX: int):
        """
        初期化 : O(MAX)
        """
        self.MAX = MAX
        isprime = self.isprime = [False] * (MAX + 1)
        mebius = self.mebius = [1] * (MAX + 1)  # メビウス関数
        min_factor = self.min_factor = [0] * (MAX + 1)  # 最小の素因数
        min_factor[1] = 1
        primes = []

        for i in range(2, MAX + 1):
            if min_factor[i] == 0:
                min_factor[i] = i
                isprime[i] = True
                mebius[i] = -1
                primes.append(i)

            smallest, mu = min_factor[i], mebius[i]
            for prime in primes:
                j = i * prime
                if j > MAX:
                    break
                min_factor[j] = prime
                if prime == smallest:
                    mebius[j] = 0
                    break
                mebius[j] = -mu

    def prime_factors(self, n: int) -> list[tuple[int, int]]:
        """
        素因数分解: (素因数, 指数) のリストを返す
        O(log n)
        """
        res = []
        while n != 1:
            prime = self.min_factor[n]
            exp = 0
            while self.min_factor[n] == prime:
                exp += 1
                n //= prime
            res.append((prime, exp))
        return res

    def divisors(self, n: int) -> list[int]:
        """
        約数列挙: n の約数をすべてリストで返す
        O(d(n)) (d(n): n の約数の個数, d(n) < O(n^(1/3)))
        """
        res = [1]
        for prime, exp in self.prime_factors(n):
            current_size = len(res)
            for i in range(current_size):
                value = 1
                for _ in range(exp):
                    value *= prime
                    res.append(res[i] * value)
        return res
```

## 6.5 Polygon.py

```
eps = 1e-10

# contains の返り値
# _OUT: 外部, _ON: 辺上, _IN: 内部
_OUT = 0
_ON = 1
_IN = 2

Point = tuple[float, float]
Polygon = list[Point]


# ベクトル p - q
def sub(p: Point, q: Point) -> Point:
    return p[0] - q[0], p[1] - q[1]


# 内積
def dot(p: Point, q: Point) -> float:
    return p[0] * q[0] + p[1] * q[1]


# 外積
def cross(p: Point, q: Point) -> float:
    return p[0] * q[1] - p[1] * q[0]


# 多角形 q と点 p の包含判定
# _OUT: 外部, _ON: 辺上, _IN: 内部
def contains(q: Polygon, p: Point) -> int:
    inside = False
    n = len(q)
    for i in range(n):
        a = sub(q[i], p)
        b = sub(q[(i + 1) % n], p)
        if a[1] > b[1]:
            a, b = b, a
        if a[1] <= eps and eps < b[1] and cross(a, b) < -eps:
            inside = not inside
        if abs(cross(a, b)) < eps and dot(a, b) <= eps:
            return _ON
    return _IN if inside else _OUT
```

## 6.6 Polygon_int.py

```
# contains の返り値
# _OUT: 外部, _ON: 辺上, _IN: 内部
_OUT = 0
_ON = 1
_IN = 2

Point = tuple[int, int]
Polygon = list[Point]


# ベクトル p - q
def sub(p: Point, q: Point) -> Point:
    return p[0] - q[0], p[1] - q[1]


# 内積
def dot(p: Point, q: Point) -> int:
    return p[0] * q[0] + p[1] * q[1]


# 外積
def cross(p: Point, q: Point) -> int:
    return p[0] * q[1] - p[1] * q[0]


# 多角形 q と点 p の包含判定
# _OUT: 外部, _ON: 辺上, _IN: 内部
def contains(q: Polygon, p: Point) -> int:
    if not q:
        return _OUT
    inside = False
    px, py = p
    prev = q[-1]
    for cur in q:
        ax, ay = prev[0] - px, prev[1] - py
        bx, by = cur[0] - px, cur[1] - py
        prev = cur
        if ay > by:
            ax, bx = bx, ax
            ay, by = by, ay
        cr = ax * by - ay * bx
        if ay <= 0 < by and cr < 0:
            inside = not inside
        if cr == 0 and ax * bx + ay * by <= 0:
            return _ON
    return _IN if inside else _OUT
```

## 6.7 Triangle.py

```
def triangle_area2(a: tuple[int, int], b: tuple[int, int], c: tuple[int, int]):
    """
    三角形 ABC の面積の 2 倍を返す。
    座標が整数なら結果も整数。
    """
    ax, ay = a
    bx, by = b
    cx, cy = c
    return abs((bx - ax) * (cy - ay) - (by - ay) * (cx - ax))


def triangle_area(a: tuple[int, int], b: tuple[int, int], c: tuple[int, int]):
    """
    三角形 ABC の面積を返す。
    """
    return triangle_area2(a, b, c) / 2
```

## 6.8 calc.py

```python
# 切り上げ除算
def ceilDiv(p: int, q: int) -> int:
    return (p + q - 1) // q


# 等差数列の和
# 初項 a1, 項数 n, 公差 d
def cal(a1, n, d):
    if n == 0:
        return 0
    return (n * (2 * a1 + (n - 1) * d)) // 2
```

## 6.9 circle_int.py

```python
Point = tuple[int, int]
Line = tuple[Point, Point]
Circle = tuple[Point, int]


# 符号を返す。正: 1, 負: -1, 0: 0
def sgn(x: int) -> int:
    if x > 0:
        return 1
    if x < 0:
        return -1
    return 0


# 2 点・2 ベクトルが等しいか
def point_eq(p: Point, q: Point) -> bool:
    return p[0] == q[0] and p[1] == q[1]


# set/sort 用の辞書順比較
def point_lt(p: Point, q: Point) -> bool:
    if p[0] != q[0]:
        return p[0] < q[0]
    return p[1] < q[1]


# ベクトル p + q
def add(p: Point, q: Point) -> Point:
    return p[0] + q[0], p[1] + q[1]


# ベクトル p - q
def sub(p: Point, q: Point) -> Point:
    return p[0] - q[0], p[1] - q[1]


# ベクトル p の整数 k 倍
def mul(p: Point, k: int) -> Point:
    return p[0] * k, p[1] * k


# 直線・線分 l の方向ベクトル B - A
def vec(l: Line) -> Point:
    return sub(l[1], l[0])


# 内積
def dot(p: Point, q: Point) -> int:
    return p[0] * q[0] + p[1] * q[1]


# 外積
def cross(p: Point, q: Point) -> int:
    return p[0] * q[1] - p[1] * q[0]


# ベクトルの長さの 2 乗
def norm(p: Point) -> int:
    return dot(p, p)


# 2 点間距離の 2 乗
def dist2(p: Point, q: Point) -> int:
    return norm(sub(p, q))


# 反時計回りに 90 度回転
def rotate90(p: Point) -> Point:
    return -p[1], p[0]


# 点 c が有向線分 ab に対してどの位置にあるか
# 1: 反時計回り, -1: 時計回り, 2: a-b-c, -2: c-a-b, 0: a-c-b
def ccw(a: Point, b: Point, c: Point) -> int:
    ab = sub(b, a)
    ac = sub(c, a)
    cr = cross(ab, ac)
    if cr > 0:
        return 1
    if cr < 0:
        return -1
    if dot(ab, ac) < 0:
        return 2
    if norm(ab) < norm(ac):
        return -2
    return 0


# 点 p が直線 l 上にあるか
def point_on_line(p: Point, l: Line) -> bool:
    return cross(sub(p, l[0]), vec(l)) == 0


# 点 p が線分 l 上にあるか
def point_on_segment(p: Point, l: Line) -> bool:
    return point_on_line(p, l) and dot(sub(p, l[0]), sub(p, l[1])) <= 0


# 2 直線が平行か
def is_parallel(l1: Line, l2: Line) -> bool:
    return cross(vec(l1), vec(l2)) == 0


# 2 直線が直交するか
def is_orthogonal(l1: Line, l2: Line) -> bool:
    return dot(vec(l1), vec(l2)) == 0


# 2 線分が交差するか。端点で接する場合も True
def segment_intersect(l1: Line, l2: Line) -> bool:
    return (
        ccw(l1[0], l1[1], l2[0]) * ccw(l1[0], l1[1], l2[1]) <= 0
        and ccw(l2[0], l2[1], l1[0]) * ccw(l2[0], l2[1], l1[1]) <= 0
    )


def radius2(c: Circle) -> int:
    assert c[1] >= 0
    return c[1] * c[1]


# 点 p と円 c の位置関係。-1: 内部, 0: 円周上, 1: 外部
def point_circle_relation(p: Point, c: Circle) -> int:
    return sgn(dist2(p, c[0]) - radius2(c))


# 点 p が円 c の円周上にあるか
def point_on_circle(p: Point, c: Circle) -> bool:
    return point_circle_relation(p, c) == 0


# 点 p が円 c の内部または円周上にあるか
def point_in_circle(p: Point, c: Circle) -> bool:
    return point_circle_relation(p, c) <= 0


# 直線 l と円 c の交点数。ゼロ長直線の場合は assert で落とす
def line_circle_intersection_count(l: Line, c: Circle) -> int:
    v2 = norm(vec(l))
    assert v2 > 0
    cr = cross(sub(c[0], l[0]), vec(l))
    x = cr * cr - radius2(c) * v2
    if x > 0:
        return 0
    return 1 if x == 0 else 2


# 直線 l と円 c が交わるか。接する場合も True
def line_circle_intersect(l: Line, c: Circle) -> bool:
    return line_circle_intersection_count(l, c) > 0


# 線分 l と円 c の円周が交わるか。端点で接する場合も True
def segment_circle_intersect(l: Line, c: Circle) -> bool:
    r2 = radius2(c)
    d0 = dist2(l[0], c[0])
    d1 = dist2(l[1], c[0])
    if l[0] == l[1]:
        return d0 == r2
    if d0 == r2 or d1 == r2:
        return True
    if d0 < r2 and d1 < r2:
        return False
    if (d0 < r2) != (d1 < r2):
        return True

    v = vec(l)
    v2 = norm(v)
    assert v2 > 0
    if dot(sub(c[0], l[0]), v) < 0 or dot(sub(c[0], l[1]), mul(v, -1)) < 0:
        return False
    cr = cross(sub(c[0], l[0]), v)
    return cr * cr <= r2 * v2


# 2 円 c1, c2 の交点数。完全に一致する場合は -1 を返す
def circle_intersection_count(c1: Circle, c2: Circle) -> int:
    center1, radius1 = c1
    center2, radius2_ = c2
    assert radius1 >= 0 and radius2_ >= 0
    d2 = dist2(center1, center2)
    if d2 == 0:
        return -1 if radius1 == radius2_ else 0

    diff2 = abs(radius1 - radius2_) ** 2
    sum2 = (radius1 + radius2_) ** 2
    if d2 < diff2 or sum2 < d2:
        return 0
    if d2 == diff2 or d2 == sum2:
        return 1
    return 2


# 2 円 c1, c2 の円周が交わるか。完全に一致する場合も True
def circle_intersect(c1: Circle, c2: Circle) -> bool:
    return circle_intersection_count(c1, c2) != 0


# 2 円 c1, c2 の共通接線の本数。完全に一致する場合は -1 を返す
def common_tangent_count(c1: Circle, c2: Circle) -> int:
    center1, radius1 = c1
    center2, radius2_ = c2
    assert radius1 >= 0 and radius2_ >= 0
    d2 = dist2(center1, center2)
    if d2 == 0 and radius1 == radius2_:
```

```
        return -1

    diff2 = abs(radius1 - radius2_) ** 2
    sum2 = (radius1 + radius2_) ** 2
    if d2 < diff2:
        return 0
    if d2 == diff2:
        return 1
    if d2 < sum2:
        return 2
    if d2 == sum2:
        return 3
    return 4
```

## 6.10 extGCD.py

```
# 拡張ユークリッドの互除法，一次合同式

# ax + by = gcd(a,b) を満たす (x,y) を求める
# 返り値：gcd(a,b), x, y
def extGCD(a, b) -> tuple[int, int, int]:
    if b == 0:
        return a, 1, 0
    g, s, t = extGCD(b, a % b)
    # g = b * s + (a%b) * t
    #   = b * s + (a - a//b * b) * t
    #   = a * t + b * s + (-a//b * b * t)
    #   = a * t + b * (s - a//b * t)
    x = t
    y = s - a // b * t
    return g, x, y


# ax ≡ 1 (mod m) を満たす x (a の逆元) を求める
def modinv(a, m) -> int:
    g, x, y = extGCD(a, m)
    assert g == 1, "a と m が互いに素でない"
    return x % m


# ax ≡ b (mod m) を満たす x を求める
def modlin(a, b, m) -> int:
    g, x, y = extGCD(a, m)
    if b % g != 0:
        return None  # 解なし
    # a*x + m*y = g より、x は a/g の mod m/g における逆元
    return x * (b // g) % (m // g)


# 中国剰余定理
# x ≡ B[i] (mod M[i]) を満たす x ≡ r (mod lcm(M)) を求める
# 返り値：解 r, lcm(M)
def chineseRem(B: list[int], M: list[int]) -> tuple[int, int]:
    assert len(B) == len(M)
    n = len(B)
    r = 0
    m = 1
    for i in range(n):
        d, p, q = extGCD(m, M[i])
        if (B[i] - r) % d != 0:
            return None, None  # 解なし
        tmp = (B[i] - r) // d * p % (M[i] // d)
        r += m * tmp
        m *= M[i] // d
    return r % m, m
```

## 6.11 factorization.py

```
from collections import defaultdict

"""
高速素因数分解 https://atcoder.jp/contests/abc177/editorial/82
素因数分解をしたい最大の数を N として
構築 O(N log log N)
素因数分解 O(log N)
init_factorization の引数に最大の数 N を入れて初期化
"""


def init_factorization(N: int) -> list[int]:
    # N 以下の整数に対して最小の素因数を求める
    # D[i] = i の最小の素因数
    D = [1] * (N + 1)
    for i in range(2, N + 1):
        if D[i] != 1:
            continue
        for j in range(i, N + 1, i):
            if D[j] == 1:
                D[j] = i
    return D


D = init_factorization()


# 素因数分解
def factorization(x: int) -> dict[int, int]:
    res = defaultdict(int)
    while x != 1:
        res[D[x]] += 1
        x //= D[x]
    return res


# エラトステネスの篩
# O (N log log N)
def eratosthenes(N: int):
    prime_number = list()  # 素数
    isprime = [True] * (N + 1)  # isprime[i]==True なら i は素数
    isprime[0], isprime[1] = False, False
    for p in range(2, N + 1):
        if not isprime[p]:
            continue
        q = p * 2
        while q <= N:
            isprime[q] = False
            q += p
    for p in range(2, N + 1):
        if isprime[p]:
            prime_number.append(p)
    return prime_number, isprime


# M 以下の素数リストを返す
# O(M log log M)
def res_primeList(M):
    isprime = [True] * (M + 1)
    isprime[0], isprime[1] = False, False
    for i in range(2, M + 1):
        if i * i > M:
            break
        if isprime[i]:
            for j in range(i * i, M + 1, i):
                isprime[j] = False
    return [i for i in range(2, M + 1) if isprime[i]]
```

## 6.12 geometry.py

```
import math


# (x1,y1) - (x2,y2) の正規化ベクトルを返す
def normalize_vector(x1, y1, x2, y2):
    dx = x1 - x2
    dy = y1 - y2
    if dx == 0:
        dx = 0
        dy = 1
    elif dy == 0:
        dx = 1
        dy = 0
    else:
        if dx < 0:
            dx *= -1
            dy *= -1
        g = math.gcd(abs(dx), abs(dy))
        dx //= g
        dy //= g
    return dx, dy


# (x0,y0) - (x1,y1) を通る直線の上に (px,py) があるか
def on_line(x0, y0, x1, y1, px, py) -> bool:
    return (y1 - y0) * (px - x0) == (x1 - x0) * (py - y0)


# 2 点 (x0, y0), (x1, y1) を通る直線 ax + by + c = 0 の (a, b, c) を返す
def get_line_abc(x0, y0, x1, y1):
    a = y1 - y0
    b = x0 - x1
    c = -(a * x0 + b * y0)
    return a, b, c


def pos_after_t(sx, sy, gx, gy, t, v, dist=None):
    """
    始点 (sx, sy) → 終点 (gx, gy) を速さ v で動くとき、
    t 秒後の位置を返す（到達したらゴールにとどまる）。
    dist = 始点と終点の距離（前計算しといた方が速い）
    """
    # (dx,dy) = 方向ベクトル
    dx = gx - sx
    dy = gy - sy
    if dist is None:
        dist = math.sqrt(dx * dx + dy * dy)

    # 始点と終点が同じ
    if dist == 0 or v <= 0:
        return sx, sy

    # 進んだ距離
    move = v * t

    # すでに（またはちょうど）到達している
    if move >= dist:
        return gx, gy

    # 全体の何割進んだか
    u = move / dist

    # 方向ベクトルに進んだ距離をかける
    x = sx + dx * u
    y = sy + dy * u
    return x, y
```

## 6.13 line__int.py

```
Point = tuple[int, int]
Line = tuple[Point, Point]


# 符号を返す。正: 1, 負: -1, 0: 0
def sgn(x: int) -> int:
    if x > 0:
        return 1
    if x < 0:
```

```
        return -1
    return 0


# 2 点・2 ベクトルが等しいか
def point_eq(p: Point, q: Point) -> bool:
    return p[0] == q[0] and p[1] == q[1]


# set/sort 用の辞書順比較
def point_lt(p: Point, q: Point) -> bool:
    if p[0] != q[0]:
        return p[0] < q[0]
    return p[1] < q[1]


# ベクトル p + q
def add(p: Point, q: Point) -> Point:
    return p[0] + q[0], p[1] + q[1]


# ベクトル p - q
def sub(p: Point, q: Point) -> Point:
    return p[0] - q[0], p[1] - q[1]


# ベクトル p の整数 k 倍
def mul(p: Point, k: int) -> Point:
    return p[0] * k, p[1] * k


# 直線・線分 l の方向ベクトル B - A
def vec(l: Line) -> Point:
    return sub(l[1], l[0])


# 内積
def dot(p: Point, q: Point) -> int:
    return p[0] * q[0] + p[1] * q[1]


# 外積
def cross(p: Point, q: Point) -> int:
    return p[0] * q[1] - p[1] * q[0]


# ベクトルの長さの 2 乗
def norm(p: Point) -> int:
    return dot(p, p)


# 2 点間距離の 2 乗
def dist2(p: Point, q: Point) -> int:
    return norm(sub(p, q))


# 反時計回りに 90 度回転
def rotate90(p: Point) -> Point:
    return -p[1], p[0]


# 点 c が有向線分 ab に対してどの位置にあるか
# 1: 反時計回り, -1: 時計回り, 2: a-b-c, -2: c-a-b, 0: a-c-b
def ccw(a: Point, b: Point, c: Point) -> int:
    ab = sub(b, a)
    ac = sub(c, a)
    cr = cross(ab, ac)
    if cr > 0:
        return 1
    if cr < 0:
        return -1
    if dot(ab, ac) < 0:
        return 2
    if norm(ab) < norm(ac):
        return -2
    return 0


# 点 p が直線 l 上にあるか
def point_on_line(p: Point, l: Line) -> bool:
    return cross(sub(p, l[0]), vec(l)) == 0


# 点 p が線分 l 上にあるか
def point_on_segment(p: Point, l: Line) -> bool:
    return point_on_line(p, l) and dot(sub(p, l[0]), sub(p, l[1])) <= 0


# 2 直線が平行か
def is_parallel(l1: Line, l2: Line) -> bool:
    return cross(vec(l1), vec(l2)) == 0


# 2 直線が直交するか
def is_orthogonal(l1: Line, l2: Line) -> bool:
    return dot(vec(l1), vec(l2)) == 0


# 2 線分が交差するか。端点で接する場合も True
def segment_intersect(l1: Line, l2: Line) -> bool:
    return (
        ccw(l1[0], l1[1], l2[0]) * ccw(l1[0], l1[1], l2[1]) <= 0
        and ccw(l2[0], l2[1], l1[0]) * ccw(l2[0], l2[1], l1[1]) <= 0
    )
```

## 6.14 mobius.py

```
def mobius(A):
    """
    高速メビウス変換
    N = log_2 |A| として O(N 2^N)
    Args:
        A: サイズが 2^N のリスト
           A[mask] には「条件 mask を少なくとも満たす組合せの総和」が入っている
    Returns:
        A: 変換後のリスト（破壊的変更）
           A[mask] には「ちょうど条件 mask だけを満たす組合せ」が入る
    """
    n = len(A).bit_length() - 1
    assert len(A) == (1 << n), "配列の長さは 2^N である必要があります"

    for i in range(n):
        bit = 1 << i
        for mask in range(1 << n):
            if not (mask & bit):
                A[mask] -= A[mask | bit]
    return A
```

## 6.15 modComb.py

```
# 二項係数ライブラリ

# 前計算 O(n + log mod)
# O(1) で nCk, nHk を返す
class Comb:
    def __init__(self, n, mod):
        self.mod = mod
        self.n = n
        size = max(2, n + 1)
        fact = self.fact = [1] * size
        factinv = self.factinv = [1] * size
        inv = self.inv = [0, 1] + [0] * (size - 2)
        for i in range(2, n + 1):
            fact[i] = fact[i - 1] * i % mod
        factinv[n] = pow(fact[n], -1, mod)
        for i in range(n, 0, -1):
            factinv[i - 1] = factinv[i] * i % mod
        for i in range(2, n + 1):
            inv[i] = factinv[i] * fact[i - 1] % mod

    def C(self, n, k):  # nCk % mod を返す
        assert n <= self.n
        if (k < 0 or n < 0) or (n < k):
            return 0
        if k == 1:
            return n
        k = min(k, n - k)
        return (
            self.fact[n] * self.factinv[k] % self.mod * self.factinv[n - k] %
            ↪  self.mod
        )

    # 重複組合せ nHr % mod を返す
    def H(self, n, r):
        return self.C(n + r - 1, r)


# O(k) 前計算
# O(min(k, n-k)) で nCk を計算（n が大きいときでも使える）
class Comb:
    def __init__(self, k, mod):
        self.k = k
        self.mod = mod
        self.fact_inv = [0] * (k + 1)
        self.inv = [0] * (k + 1)
        self.fact_inv[0] = 1
        if k >= 1:
            self.fact_inv[1] = 1
            self.inv[1] = 1

        for i in range(2, k + 1):
            self.inv[i] = mod - self.inv[mod % i] * (mod // i) % mod
            self.fact_inv[i] = self.fact_inv[i - 1] * self.inv[i] % mod

    def nCk(self, n, k):
        if k < 0 or k > n:
            return 0
        k = min(k, n - k)
        if k > self.k:
            raise ValueError("precomputed k is too small")
        mod = self.mod
        ans = 1
        for i in range(k):
            ans = ans * (n - i) % mod
        return ans * self.fact_inv[k] % mod


class PascalsTriangle:
    """
    パスカルの三角形で nCk を求める
    計算量: O(N^2)
    mod が素数でなくても使用可能
    """

    def __init__(self, N, mod):
        self.mod = mod
        self.binom = [[0] * (N + 1) for _ in range(N + 1)]
        for i in range(N + 1):
            self.binom[i][0] = 1
            for j in range(1, i + 1):
                self.binom[i][j] = (
                    self.binom[i - 1][j - 1] + self.binom[i - 1][j]
                ) % mod

    def C(self, n, k):
        assert 0 <= k <= n
        return self.binom[n][k]

    def H(self, n, k):
        """重複組合せ nHk"""
```

```
        assert n >= 0 and k >= 0
        return self.C(n + k - 1, k)
```

### 6.16 modDiv.py

```
def modDiv(P, Q, mod):  # P/Q mod m
    denominator = pow(Q, -1, mod)  # Q が同じなら前計算したほうが速い
    return P * denominator % mod


def max_leq_congruent(R: int, r: int, m: int) -> int:
    """R 以下で r ≡ x (mod m) を満たす最大の整数 x を求める"""
    r %= m
    return R - ((R - r) % m)


def min_geq_congruent(L: int, r: int, m: int) -> int:
    """L 以上で r ≡ x (mod m) を満たす最小の整数 x を求める"""
    if m <= 0:
        raise ValueError("m must be positive")
    r %= m
    return L + ((r - L) % m)
```

## 7 string

### 7.1 RollingHash.py

```
class RollingHash:
    def __init__(self, S, base=1237, mod=(1 << 61) - 1):
        N = len(S)
        ord_base = ord("a")
        self.base = base
        self.mod = mod
        self.hash = [0] * (N + 1)
        self.base_pow = [1] * (N + 1)
        for i in range(N):
            self.base_pow[i + 1] = self.base_pow[i] * base % mod
        for idx, s in enumerate(S):
            tmp = ord(s) - ord_base + 1  # 0 を割り当ててはいけない
            self.hash[idx + 1] = (self.hash[idx] * self.base + tmp) % mod

    def get_hash(self, l, r):  # [l, r)
        return (self.hash[r] - self.hash[l] * self.base_pow[r - l]) % self.mod


# マルチテストケース用
MAX = 20000 + 1
base = 1237
mod = (1 << 61) - 1
base_pow = [1] * MAX
for i in range(MAX - 1):
    base_pow[i + 1] = base_pow[i] * base % mod
hash = [0] * MAX
ord_base = ord("a")


def init(S):
    hash[0] = 0
    for idx, s in enumerate(S):
        tmp = ord(s) - ord_base + 1  # 0 を割り当ててはいけない
        hash[idx + 1] = (hash[idx] * base + tmp) % mod


def get_hash(l, r):  # [l, r)
    return (hash[r] - hash[l] * base_pow[r - l]) % mod
```

### 7.2 SuffixArray.py

```
# SA の構築：O(|S|)
# SA[i]: int で S の SA[i] から始まる suffix がソートされてる


def sa_is(s, upper):
    n = len(s)
    if n < 2:
        return list(range(n))
    ls = [0] * n
    for i in range(n - 2, -1, -1):
        ls[i] = ls[i + 1] if s[i] == s[i + 1] else s[i] < s[i + 1]
    sum_l = [0] * (upper + 1)
    sum_s = [0] * (upper + 1)
    for i in range(n):
        if not ls[i]:
            sum_s[s[i]] += 1
        else:
            sum_l[s[i] + 1] += 1
    for i in range(upper + 1):
        sum_s[i] += sum_l[i]
        if i < upper:
            sum_l[i + 1] += sum_s[i]

    def induce(lms):
        sa = [-1] * n
        buf = sum_s[:]
        for d in lms:
            sa[buf[s[d]]] = d
            buf[s[d]] += 1
        buf = sum_l[:]
        sa[buf[s[-1]]] = n - 1
        buf[s[-1]] += 1
        for v in sa:
            if v > 0 and not ls[v - 1]:
                sa[buf[s[v - 1]]] = v - 1
                buf[s[v - 1]] += 1
        buf = sum_l[:]
        for v in reversed(sa):
            if v > 0 and ls[v - 1]:
                buf[s[v - 1] + 1] -= 1
                sa[buf[s[v - 1] + 1]] = v - 1
        return sa

    lms = [i for i in range(1, n) if not ls[i - 1] and ls[i]]
    m = len(lms)
    ids = [-1] * n
    for i, v in enumerate(lms):
        ids[v] = i
    sa = induce(lms)
    if m:
        order = [v for v in sa if ids[v] != -1]
        lms.append(n)
        rec = [0] * m
        rank = 0
        # LMS 部分文字列は次の LMS 位置の文字まで含めて比較する。
        # 各区間の比較は高々 2 回なので、スライスの合計長も O(n)。
        for l, r in zip(order, order[1:]):
            el, er = lms[ids[l] + 1], lms[ids[r] + 1]
            rank += s[l : el + 1] != s[r : er + 1]
            rec[ids[r]] = rank
        sa = induce([lms[i] for i in sa_is(rec, rank)])
    return sa


def suffix_array_upper(s, upper):
    assert 0 <= upper
    for d in s:
        assert 0 <= d <= upper
    return sa_is(s, upper)


def suffix_array(s) -> list[int]:
    n = len(s)
    if type(s) == str:
        return sa_is(list(map(ord, s)), 255)
    idx = sorted(range(n), key=s.__getitem__)
    s2 = [0] * n
    now = 0
    for i, j in zip(idx, idx[1:]):
        now += s[i] != s[j]
        s2[j] = now
    return sa_is(s2, now)


# lcp[i] = S の SA[i] 文字目から始まる suffix と S の SA[i+1] 文字目から始まる suffix
# ↪ の共通接頭辞の長さ
def lcp_array(s: str, sa: list[int]) -> list[int]:
    n = len(s)
    assert n >= 1
    rnk = [0] * n
    for i, v in enumerate(sa):
        rnk[v] = i
    lcp = [0] * (n - 1)
    h = 0
    for i in range(n):
        if h:
            h -= 1
        if rnk[i] == 0:
            continue
        j = sa[rnk[i] - 1]
        while j + h < n and i + h < n and s[j + h] == s[i + h]:
            h += 1
        lcp[rnk[i] - 1] = h
    return lcp


# suffix をソートして T <= x なる suffix x のインデックスを返す
# T <= S[SA[i] : SA[i] + len(T)] を満たす最小の i
def suffix_array_bisect(S, SA, T):
    ng, ok = -1, len(S)
    while ok - ng > 1:
        mid = (ok + ng) // 2
        idx = SA[mid]
        if T <= S[idx : idx + len(T)]:
            ok = mid
        else:
            ng = mid
    return ok


# SA = suffix_array(S)
```

### 7.3 Z-algo.py

```
# O(|S|)
# Z[i] := S[0:|S|) と S[i:|S|) の最長共通接頭辞の長さ
def z_algo(S):
    n = len(S)
    Z = [0] * n
    Z[0] = n
    i, j = 1, 0
    while i < n:
        # 一致する限り延長する
        while i + j < n and S[j] == S[i + j]:
            j += 1
        Z[i] = j

        # 一文字も一致しないならコピーできないので次に進む
        if j == 0:
            i += 1
            continue

        k = 1
        # k+Z[k]>j なら 1 回目の登場の際にはみ出してしまっている
        # k+Z[k]=j なら 2 回目の登場の際にはみ出す可能性がある
        while k < j and k + Z[k] < j:
            Z[i + k] = Z[k]
            k += 1
```

```
        # まだ未確定なところまで i を進める
        i += k
        # j を i に合わせる
        j -= k

    return Z
```

## 7.4 changeable_RollingHash.py

```
class segtree:  # すべて 0-index
    def __init__(self, V, OP, E):
        self.n = len(V)
        self.op = OP
        self.e = E
        self.log = (self.n - 1).bit_length()
        self.size = 1 << self.log
        self.data = [E for i in range(2 * self.size)]
        for i in range(self.n):
            self.data[self.size + i] = V[i]
        for i in range(self.size - 1, 0, -1):
            self._update(i)

    # 1 点更新
    def set(self, p, x):
        assert 0 <= p and p < self.n
        p += self.size
        self.data[p] = x
        for i in range(1, self.log + 1):
            self._update(p >> i)

    # 1 点加算（自分で書いた）
    def add(self, p, x):
        assert 0 <= p and p < self.n
        p += self.size
        self.data[p] += x
        for i in range(1, self.log + 1):
            self._update(p >> i)

    # data[p] を返す
    def get(self, p):
        assert 0 <= p and p < self.n
        return self.data[p + self.size]

    # [l,r) の演算結果を返す
    def prod(self, l, r):
        assert 0 <= l and l <= r and r <= self.n
        sml = self.e
        smr = self.e
        l += self.size
        r += self.size
        while l < r:
            if l & 1:
                sml = self.op(sml, self.data[l])
                l += 1
            if r & 1:
                smr = self.op(self.data[r - 1], smr)
                r -= 1
            l >>= 1
            r >>= 1
        return self.op(sml, smr)

    # 全区間の演算結果を返す
    def all_prod(self):
        return self.data[1]

    # check(operate(data[l],data[l+1],...,data[r-1])) = True
    # を満たす最大の r を返す
    def max_right(self, l, check):
        assert 0 <= l and l <= self.n
        assert check(self.e)
        if l == self.n:
            return self.n
        l += self.size
        sm = self.e
        while True:
            while l % 2 == 0:
                l >>= 1
            if not (check(self.op(sm, self.data[l]))):
                while l < self.size:
                    l = 2 * l
                    if check(self.op(sm, self.data[l])):
                        sm = self.op(sm, self.data[l])
                        l += 1
                return l - self.size
            sm = self.op(sm, self.data[l])
            l += 1
            if (l & -l) == l:
                break
        return self.n

    # check(operate(data[l],data[l+1],...,data[r-1])) = True
    # を満たす最小の l を返す
    def min_left(self, r, check):
        assert 0 <= r and r <= self.n
        assert check(self.e)
        if r == 0:
            return 0
        r += self.size
        sm = self.e
        while True:
            r -= 1
            while r > 1 and (r % 2):
                r >>= 1
            if not (check(self.op(self.data[r], sm))):
                while r < self.size:
                    r = 2 * r + 1
                    if check(self.op(self.data[r], sm)):
                        sm = self.op(self.data[r], sm)
                        r -= 1
                return r + 1 - self.size
            sm = self.op(self.data[r], sm)
            if (r & -r) == r:
                break
        return 0

    def _update(self, k):
        self.data[k] = self.op(self.data[2 * k], self.data[2 * k + 1])

    def __str__(self):
        return str([self.get(i) for i in range(self.n)])


class changeable_RollingHash:
    def __init__(self, S, base=1237, mod=(1 << 61) - 1):
        N = len(S)
        self.N = N
        self.ord_base = ord("a")
        self.base = base
        self.mod = mod
        self.base_pow = [1] * (N + 1)
        self.base_inv = [1] * (N + 1)
        self.inv = pow(base, mod - 2, mod)
        for i in range(N):
            self.base_pow[i + 1] = (self.base_pow[i] * base) % mod
            self.base_inv[i + 1] = (self.base_inv[i] * self.inv) % mod
        hash = [
            ((ord(s) - self.ord_base + 1) * self.base_pow[N - idx - 1]) % mod
            for idx, s in enumerate(S)
        ]
        e = 0

        def operate(a, b):
            return (a + b) % mod

        self.seg = segtree(hash, operate, e)

    # インデックス x の要素を c に変える
    def change(self, x, c):
        self.seg.set(
            x, ((ord(c) - self.ord_base + 1) * self.base_pow[self.N - x - 1]) %
            ↪  self.mod
        )

    def get_hash(self, l, r):  # [l, r),0-index
        return (self.seg.prod(l, r) * self.base_inv[self.N - r]) % self.mod
```

## 7.5 sort_by_concat.py

```
# O(|S|)
# Z[i] := S[0:|S|) と S[i:|S|) の最長共通接頭辞の長さ
def z_algo(S):
    n = len(S)
    Z = [0] * n
    Z[0] = n
    i, j = 1, 0
    while i < n:
        # 一致する限り延長する
        while i + j < n and S[j] == S[i + j]:
            j += 1
        Z[i] = j

        # 一文字も一致しないならコピーできないので次に進む
        if j == 0:
            i += 1
            continue

        k = 1
        # k+Z[k]>j なら 1 回目の登場の際にはみ出してしまっている
        # k+Z[k]=j なら 2 回目の登場の際にはみ出す可能性がある
        while k < j and k + Z[k] < j:
            Z[i + k] = Z[k]
            k += 1
        # まだ未確定なところまで i を進める
        i += k
        # j を i に合わせる
        j -= k

    return Z


# reference:
↪  https://info.atcoder.jp/entry/algorithm_lectures/lex_min_string_concatenation
def sort_by_concat(T: list[str]) -> list[str]:
    """
    文字列配列 T = [S_1, S_2, ...] の並べ替えであり、文字列を結合 S_1 + S_2 + ... したとき辞書順最小になる順番に並べ替えて返す
↪
    S_i+S_j を基準にソートすればいい
    T の辞書順ソートではない
    """

    # Z-array は比較で必要になったときに一度だけ構築する
    Z = [None] * len(T)

    # a, b の辞書順を返す
    def lexcmp(a: str, b: str) -> int:
        return (a > b) - (a < b)

    def cmp(x: int, y: int) -> int:
        """
        a+b < b+a なら -1, a+b == b+a なら 0, a+b > b+a なら 1
        """
        a, b = T[x], T[y]
        isrev = len(a) > len(b)
        if isrev:
            a, b = b, a
            x, y = y, x
        n, m = len(a), len(b)

        # 1. a と b[:n]
        f = lexcmp(a, b[:n])
```

```
61
62          # 2. b[:m-n] と b[n:m]
63          if f == 0 and n < m:
64              zb = Z[y]
65              if zb is None:
66                  Z[y] = zb = z_algo(b)
67              k = m - n
68              lcp = min(zb[n], k)
69              # b[:m-n] != b[n:m] なら lcp の直後の文字で辞書順決まる
70              if lcp < k:
71                  f = lexcmp(b[lcp], b[n + lcp])
72
73          # 3. b[m-n:] と a
74          if f == 0:
75              f = lexcmp(b[m - n :], a)
76
77          return -f if isrev else f
78
79      def merge_sort(arr: list[int]):
80          # arr の要素は T のインデックス
81          if len(arr) <= 1:
82              return arr
83
84          mid = len(arr) // 2
85          left = merge_sort(arr[:mid])
86          right = merge_sort(arr[mid:])
87
88          res = []
89          i = j = 0
90
91          while i < len(left) and j < len(right):
92              if cmp(left[i], right[j]) <= 0:
93                  res.append(left[i])
94                  i += 1
95              else:
96                  res.append(right[j])
97                  j += 1
98
99          res.extend(left[i:])
100         res.extend(right[j:])
101         return res
102
103     return [T[i] for i in merge_sort(list(range(len(T))))]
```

# 8 techniques

## 8.1 3buntansaku.py

```
1  def ternary_search_float(f, left, right, eps=1e-9, maximize=False):
2      """
3      f が [left, right] 上で 凸 or 凹 だと仮定して、
4      その最小値（または最大値）を 3 分探索で求める。
5      maximize=False: 最小値を探す
6      maximize=True : 最大値を探す
7      返り値: (最適解の範囲の左端, 右端, f(中点))
8      """
9      while right - left > eps:
10         m1 = (2 * left + right) / 3
11         m2 = (left + 2 * right) / 3
12         f1 = f(m1)
13         f2 = f(m2)
14
15         if not maximize:
16             if f1 > f2:
17                 left = m1
18             else:
19                 right = m2
20         else:
21             if f1 < f2:
22                 left = m1
23             else:
24                 right = m2
25
26         if right - left < eps:
27             break
28
29     x = (left + right) / 2
30     return left, right, f(x)
31
32
33 def ternary_search_int(f, left, right, eps, maximize=False):
34     """
35     f: int -> 値 を返す一峰性関数（山 or 谷）
36     [left, right] の整数範囲で、f の最大値 or 最小値を探す。
37     maximize=True  なら最大値を探す
38     maximize=False なら最小値を探す
39
40     戻り値: (arg_opt, f(arg_opt))
41     """
42
43     while right - left > eps:
44         m1 = (2 * left + right) // 3
45         m2 = (left + 2 * right) // 3
46
47         f1 = f(m1)
48         f2 = f(m2)
49
50         if not maximize:
51             if f1 > f2:
52                 left = m1
53             else:
54                 right = m2
55         else:
56             if f1 < f2:
57                 left = m1
58             else:
59                 right = m2
60
61     # 残った [left, right] を愚直に見る
62     best_x = left
63     best_val = f(left)
64     for x in range(left + 1, right + 1):
65         val = f(x)
66         if (maximize and val > best_val) or (not maximize and val < best_val):
67             best_val = val
68             best_x = x
69
70     return best_x, best_val
```

## 8.2 CmpSort.py

```
1  def cmp_sort(a, cmp):
2      """比較関数 cmp に従ってリスト a を破壊的に安定ソートする。
3
4      cmp(x, y) は x が先なら負、同順位なら 0、後なら正を返す。
5      比較結果は一貫した順序を定めること。並べ替えた同じリスト a を返す。
6      時間 O(n log n)、追加領域 O(n)。整列済み・狭義逆順なら O(n)。
7      PyPy 向けに比較関数を直接呼び出す、非再帰のマージソート。
8      """
9      n = len(a)
10     # 全体が昇順または狭義降順なら O(n) で終了する。
11     if n > 1:
12         d = cmp(a[0], a[1]) > 0
13         r = 2
14         while r < n and (cmp(a[r - 1], a[r]) > 0) == d:
15             r += 1
16         if r == n:
17             if d:
18                 a.reverse()
19             return a
20
21     # 幅 w の整列済み区間を隣同士でマージする。
22     w = 1
23     while w < n:
24         for l in range(0, n - w, 2 * w):
25             m, r = l + w, min(l + 2 * w, n)
26             if cmp(a[m - 1], a[m]) <= 0:
27                 continue
28             b = a[l:m]  # 左側だけ退避する。長さは常に w。
29             i, j, k = 0, m, l
30             while i < w and j < r:
31                 if cmp(b[i], a[j]) <= 0:
32                     a[k] = b[i]
33                     i += 1
34                 else:
35                     a[k] = a[j]
36                     j += 1
37                 k += 1
38             a[k:j] = b[i:]
39         w *= 2
40     return a
```

## 8.3 Compression.py

```
1  # 座標圧縮
2  # ex. A=[2,9,9,7,9,2,4] -> 返り値=[1,4,4,3,4,1,2]
3  def compression(A):
4      B = [0] * len(A)
5      Adic = {x: i for i, x in enumerate(sorted(list(set(A))))}
6      for i in range(len(A)):
7          B[i] = Adic[A[i]]
8      return B
```

## 8.4 CrossAccumulate.py

```
1  class CrossAccumulate:
2      def __init__(self, grid: list[list[int]]):
3          self.H = len(grid)
4          self.W = len(grid[0])
5          self.s = [[0] * (self.W + 1) for _ in range(self.H + 1)]
6
7          s = self.s
8          for i in range(self.H):
9              prev, cur, row = s[i], s[i + 1], grid[i]
10             total = 0
11             for j in range(self.W):
12                 total += row[j]
13                 cur[j + 1] = prev[j + 1] + total
14
15     def query(self, r1: int, c1: int, r2: int, c2: int) -> int:
16         """
17         左上 (r1,c1), 右下 (r2,c2) の閉区間矩形の和を返す
18         """
19         assert 0 <= r1 <= r2 < self.H and 0 <= c1 <= c2 < self.W
20         return (
21             self.s[r2 + 1][c2 + 1]
22             - self.s[r1][c2 + 1]
23             - self.s[r2 + 1][c1]
24             + self.s[r1][c1]
25         )
```

## 8.5 Doubling.py

```
1  def doubling(dp, LEN, POW):
2      for i in range(1, POW):
3          prev, cur = dp[i - 1], dp[i]
4          for j in range(LEN):
5              cur[j] = prev[prev[j]]
6      return dp
7
8
9  def query(dp, j, k):
10     """
```

```
    j から始めて k 回移動する
    """
    tmp = j
    while k:
        i = k.bit_length() - 1
        tmp = dp[i][tmp]
        k ^= 1 << i
    return tmp
```

## 8.6 MegruBisect.py

```
def megru_bisect(ng, ok):
    def judge(mid):
        return

    while abs(ok - ng) > 1:
        mid = (ok + ng) // 2
        if judge(mid):  # 条件を満たすなら
            ok = mid
        else:
            ng = mid

    return ng, ok
```

## 8.7 Rotate.py

```
def two_dimension_list_rotate_R90(L: list[list[int]]) -> list[list[int]]:
    # 2 次元リストを右に 90 度回転させる
    return [list(col) for col in zip(*L[::-1])]


def two_dimension_list_rotate_L90(L: list[list[int]]) -> list[list[int]]:
    # 2 次元リストを左に 90 度回転させる
    return [list(col) for col in zip(*L)][::-1]


def tention(L: list[list[int]]) -> list[list[int]]:
    # 2 次元リストを転置する
    return [list(col) for col in zip(*L)]
```

## 8.8 RunLengthCompression.py

```
def run_length_compression(S: str):
    res = []
    tmp = S[0]
    it = iter(S)
    next(it)
    cnt = 1
    for s in it:
        if s == tmp:
            cnt += 1
        else:
            res.append((tmp, cnt))
            tmp = s
            cnt = 1
    res.append((tmp, cnt))
    return res
```

## 8.9 c__is__buween__a__to__b.py

```
# https://atcoder.jp/contests/abc367/editorial/10712
def c_is_buween_a_to_b(a, b, c):
    return (a - b) * (b - c) * (c - a) > 0
```

## 8.10 imos.py

```
from itertools import accumulate


# 二次元の imos 法
class imos:
    def __init__(self, H: int, W: int):
        self.grid = [[0] * W for _ in range(H)]

    def add(self, i0: int, j0: int, i1: int, j1: int, increment: int) -> None:
        """
        左上を (i0,j0), 右下を (i1,j1) とする長方形領域に increment 加算する

        grid[i0][j0], grid[i0][j0+1], ..., grid[i0][j1]
        grid[i0+1][j0], grid[i0+1][j0+1], ..., grid[i0+1][j1]
        ...
        grid[i1][j0], grid[i1][j0+1], ..., grid[i1][j1]
        """
        assert 0 <= i0 <= i1 < len(self.grid) and 0 <= j0 <= j1 <
        ↪  len(self.grid[0])

        self.grid[i0][j0] += increment
        if i1 + 1 < len(self.grid):
            self.grid[i1 + 1][j0] -= increment
        if j1 + 1 < len(self.grid[0]):
            self.grid[i0][j1 + 1] -= increment
        if i1 + 1 < len(self.grid) and j1 + 1 < len(self.grid[0]):
            self.grid[i1 + 1][j1 + 1] += increment

    # 二次元累積和を取る
    def cross_accumulate(self):
        self.flag = True
        grid = self.grid
        H, W = len(grid), len(grid[0])
        grid[0] = list(accumulate(grid[0]))
        for i in range(1, H):
            prev, cur = grid[i - 1], grid[i]
            total = 0
            for j in range(W):
                total += cur[j]
                cur[j] = prev[j] + total
        return grid
```